workforce SOFTWARE | MAKING WORK EASY

REST API GUIDE

WorkForce Software Suite

January 2025

LEGAL NOTICES

Copyright (c) 2000-2025 WorkForce Software, LLC. All rights reserved.

WorkForce Software
38705 Seven Mile Rd.
Livonia, MI 48152
United States

Email: info@workforcesoftware.com
Website: https://www.workforcesoftware.com

Phone: +1-877-493-6723

Workforce Software considers the enclosed information a trade secret. By receiving this information, you agree to keep this information confidential. This information may not be distributed outside your organization. It may not be duplicated in any way without the express written consent of WorkForce Software, except that you are given permission to duplicate it in electronic or printed form for the purpose of distribution within your organization to gather requirements or evaluate our software.

The information supplied is distributed on an “as is” basis, without any warranty. WorkForce Software shall not have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained herein.

Trademark names may appear throughout this material. Rather than list the names and entities that own those trademarks or insert a trademark symbol with each mention of the trademarked name, WorkForce Software states that it is using those names in editorial fashion only for the benefit of the trademark owner, with no intention of infringing on the trademark. No mention of a company or trademark is intended to convey endorsement or other affiliation with WorkForce Software.

If you have been provided this document under any other circumstances, you must contact WorkForce Software at the following phone number to arrange to have this material returned immediately: +1-877-493-6723.

This document was last updated on 7 January 2025.

Contents

Change Log 6
Overview 7
Getting Started 7
Prerequisites 7
OpenAPI Specifications 7
Authentication and Authorization 7
How should I create a Salesforce ticket? 8
Making a Request 9
Response Messages 9
Data Types 10
Behavior of REST interfaces 10
API Versions 11
Data Feed APIs 11
Asynchronous Processing APIs 11
How to Authenticate an API Call 11
How to Call a Data Feed API 13
How to Call Standard Application APIs 14
REST API Endpoints 17
Bank 17
GET /bank/v1 17
Bank Events 18
GET /bank-events/v1 18
Calculated Time 19
GET /calculated-time/v1 19
GET /calculated-time/employee/v1 20
Clock 21
GET /clock/v1 21
Clock 22
PUT /clock/v1 22

Data Translation 23
GET /data-translation/v1 23
Employee 24
GET /employee/v1 24
Employee Group 25
GET /employee-group/v1 25
Errors and Warnings 26
GET /errors-and-warnings/v1 26
Pay 27
GET /pay/v1 27
Person 28
GET /person/v2 28
PUT /person/v2 28
How To Use this API 28
Policy Set 50
GET /policy-set/v1 50
Role Detail 50
GET /role-detail/v1 50
Shift 51
GET /shift/v1 51
GET /shift/employee/v1 51
Swipe 51
GET /swipe/v1 51
Time 52
GET /time/v1 52
PUT /time/v2 52
Time Exceptions 53
GET /time-exceptions/v1 53
Time Off 53
GET /time-off/v1 53

GET /time-off/employee/v1 53
PUT /time-off/v1 54
Time Off Request 54
GET /time-off-request/v1 54
Timesheet Summary 54
GET /timesheet-summary/v1 54
User 55
GET /user/v1 55
User Rights 55
GET /user-right/v2 55
Create a Data Store 56
Overview 56
Before You Get Started 56
Getting Started 57
Data Model 58
Employee 59
EmployeeJob 64
CalcTime 68
Pay 68
PayCode 69
Swipe 70
TimeOff 70
User 71
UserRights 72
Using the Data Enrichment API 73
Common Use Cases 78
Testing an Enrichment Definition 82
Appendix A: Field Mappings for Person v2 84
Appendix B: Frequently Asked Questions (FAQs) 107

Change Log

Version	Date	Description
January 2024	11 November 2023	New version removing anything already covered by OpenAPI Specification, added Data Enrichment section, plus other minor improvements.
May 2024	13 May 2024	Added new section: Create a Data Store | Before You Get Started. Also, other improvements.
August 2024	29 August 2024	Updated fields for Address in Appendix A.
January 2025	7 January 2025	Update copyright date for the new year.

Overview

WorkForce Software's REST APIs enable customers to send to and receive data from the WorkForce Integration platform. Client developers can set up a seamless integration between their own systems and the WorkForce Suite. Our modern, advanced APIs use commonly understood technology and techniques so the WFS platform can be easily adopted into your API ecosystem.

Getting Started

The following sections provide the information needed to easily get started in using the WorkForce Software REST APIs.

Prerequisites

Your account must meet the following criteria before getting started with using WorkForce REST APIs:

- WorkForce Software v20.2 or later
- WorkForce Suite enabled
- Connection to the OCI data center (data feed only)
    - This requires the Data Feeds API feature flag to be enabled, which must be requested through a Salesforce ticket.

OpenAPI Specifications

Specifications of all APIs are available in OpenAPI format. These specifications can be found as a Swagger UI web document at docs.integration.wfs.cloud. The specifications provide definitions of all WorkForce public APIs. If you are experienced with REST APIs, the OpenAPI spec may be all you need. If you need further information, e.g., how to achieve certain outcomes, then this document can provide more context surrounding API use.

Authentication and Authorization

You will authenticate all API calls made over HTTPS using OAuth 2.0 protocols.

To call an API to retrieve (GET) or push (PUT) data, you must first call the Auth API using your Secret Key. This retrieves an Access Token which is then used in an HTTP GET or PUT request to the API. Data is then returned in JSON format as specified for each API.

You must request a Secret Key from WorkForce Support via Salesforce. Secret keys are issued for an associated client id. Each client id can be configured for single role or a group of roles.

For example, you may have a client id called clock-connector that is configured to give access to the Clock API only. API keys generated for the clock-connector will only give access to the Clock API. You might also have a client id called person-connector which gives access to the Person API.

Additionally, you will need the CAS-URL and API-URL for your instance. These are of the form `cas-<region>.wfs.cloud` and `api-<region>.wfs.cloud` respectively. The region refers to the regional data center (i.e., EU3, AU3, US8).

The URL for a REST API call will have the following pattern:

`https://api-<region>.wfs.cloud/<apiName>/<version>?<params>`

So, an example call may look like the following:

- `https://api-eu3.wfs.cloud/bank/v1?count=1`
- https://cas-au3.wfs.cloud/aug/api/v1.0/authenticate/client/<ACCOUNT>?client_id=serviceaccount&client_secret=<secret`https://cas-au3.wfs.cloud/auth/realms/<ACCOUNT>/protocol/openid-connect/token`
- `https://api-au3.wfs.cloud/timesheet-summary/v1?externalMatchId=<employeeId>`

Once you have the secret key and the additional URL configuration, you will be able to get up and running quickly, either by using a client such as Postman or your own preferred tools.

How should I create a Salesforce ticket?

Create a new case in Salesforce to obtain your Secret Key or to request that the Data Feeds API feature flag is enabled. Include the information:

- Subject:
  - For the Secret Key request: I would like the client Id, account, and secret for using REST APIs with <ACCOUNT>. (Replace <ACCOUNT> with the customer name.)
  - For the Feature Flag request: I would like the Data Feed API feature flag enabled for using data feed REST APIs with <ACCOUNT>. (Replace <ACCOUNT> with the customer name.)
- Description: enter any needed info
- Case Topic: WorkForce Products
- Service Area: WorkForce Suite
- Service: Suite Authentication
- Request: Suite Authentication
- Case Type: Question

- Account Name: customer name

Making a Request

When you make a basic request using a REST API, there is information that should be included to ensure a successful response.

The header should contain the following values:

- **Content-Type** – set to application/json
- **Authorization** – needed for authentication. This is retrieved by making a request to the Authentication API using your Secret Key. Should be prepended with the word "Bearer" and a space.

The following parameters may also be included for Data Feed APIs:

- **Cursor** - to say from how far back to stream data. A null cursor will return data from the start of the feed.
- **Count** – to specify the maximum number of records to return. The count is set to 2,000 by default and has a maximum of 10,000.

Response Messages

The API responses use traditional HTTP response codes supplied in Status Code HTTP header.

Type	HTTP Response Code	Reason
Request successfully received	200	OK Success
Authentication	403	Forbidden
Request format	400	Bad Request
Requested resource not found	404	Not found
Internal Error	500	Internal Server Error

Data Types

Name	Description
Date	ISO-8601-formatted date
Email	Properly formatted email address. i.e., local-name@domain
Enum	Restricted selection
String	Any characters
Boolean	true | false
Number	Decimal or whole numeric value
Integer	Whole number which can be negative, positive or zero
Country Code	ISO-3166-1 alpha-3 country codes
State/Province	ISO 3166-2 country sub-division codes
TZ	IANA Time Zone
Phone	E164 formatted telephone number

Behavior of REST interfaces

WorkForce APIs follow regular HTTP verb semantics.

PUT replaces the resource referenced in the request with the resource contained within the request body. PUT messages are for a single instance of an entity; the Person API takes one Person per request. It is used to both create and update the state of a resource on the server. For example, sending details of a person who does not exist in the system to the Person API will result in a new employee/user. If the person does exist in the system, then the details we currently hold are replaced by the ones you send. Where date effective records are supported, it is possible to send just the latest data and not be required to always send full history.

GET is used to retrieve data and does not involve any changes to data held in the system. A call to a resource without any parameters will return all data (up to a max). The cursor parameter is used to indicate where in the data stream to retrieve from. The count parameter is used to

indicate the number of records you want in the response. The maximum number of records pulled by Data Feeds is 10,000 with a default of 2,000.

GET and PUT methods are safe to repeat with the same payload. Concurrent requests to individual APIs are supported (except for Data Feeds).

API Versions

The WorkForce REST APIs versions are long lived. As further features are added to the APIs, this is always done in such a way that is backwards compatible, unless a breaking change is unavoidable. Breaking changes will be released as a new version. Concurrent versions will be supported with older versions eventually being deprecated and ultimately removed as appropriate and in consultation with customers.

Data Feed APIs

Data Feed API are only available to customers who have been implemented in an OCI data center. If you have not yet been migrated, this is a pre-requisite of using Data Feeds.

Data Feed APIs are intended to be called frequently to supply real-time data. These APIs return an array of JSON objects, each with an identifier, along with a cursor to identify how far along the data stream the API client has reached. Data Feed APIs allow the cursor as an optional parameter. However, in normal use, clients are expected to provide the cursor returned from the previous call, allowing the client to continuously receive a data feed as it is modified or created by WorkForce Software applications.

The Data feed API feature flag must be enabled. There must be a separate request for each environment.

Asynchronous Processing APIs

APIs will respond immediately with any general problems with the request. There are also more detailed checks that occur after the API request has been received and a response sent. For example, a data validation failure, which the client needs to know about. As the request is completed, the API responds with the HTTP response codes shown earlier. To receive details of any errors generated during asynchronous processing, developers must supply an email address. An error reporting mechanism which supports other channels besides email is on our road map.

How to Authenticate an API Call

For all APIs, you will first need to call the Authentication Service to retrieve an `Authorization access token`. This can be done using a POST request like the following:

```
POST
 https://{{host.cas}}/auth/realms/{{account}}/protocol/openid-connect/token
```

As described above `host.cas` will be in the form `cas-dc.wfs.cloud`, where `dc` is the data center (for example, US2, EU1, AU3, CA1, etc.). The `account` is referring to the External Account Id from Tenant Manager.

For this request, a content type of `application/x-www-form-urlencoded` must be used with a body containing three parameters:

- grant_type - will always be `client_credentials`
- client_id - should be obtained securely through Customer Support
- client_secret. - should be obtained securely through Customer Support

Making this call returns a response like this.


```
{"access_token":"eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJaS0JjT1dBb3RhT09hQ1h6WEI0N
0ZPeVNGQmJMeEMyWm1vSUE1U0VwbWFJIn0.eyJleHAiOjE2ODQ0MTYxODksImlhdCI6MTY4NDQxNTg4OSwianRpIjoiMGV
mNmJkYmQtMjIzYS00OTFkLWJmN2YtZGQ0MmVhMTMxYThkIiwiaXNzIjoiaHR0cHM6Ly9jYXMtdXM4Lndmcy5jbG91ZC9hd
XRoL3JlYWxtcy93ZnNfZW5nX3ZhbGlkX3VzOCIsImF1ZCI6ImFjY291bnQiLCJzdWIiOiIwMDBlYzJhOC0zNzQ4LTRjZWU
tYjE4MC1kNDg2YTUxNWY1NDIiLCJ0eXAiOiJCZWFyZXIiLCJhenAiOiJzZXJ2aWNlYWNjb3VudCIsInNlc3Npb25fc3Rhd
GUiOiI0ZWQ3MGJlZi0zYjg4LTQ1MWItOTg4YS1hMWE5ODRkYzhmOTIiLCJhY3IiOiIxIiwicmVhbG1fYWNjZXNzIjp7InJ
vbGVzIjpbIm9mZmxpbmVfYWNjZXNzIiwidW1hX2F1dGhvcml6YXRpb24iXX0sInJlc291cmNlX2FjY2VzcyI6eyJhY2Nvd
W50Ijp7InJvbGVzIjpbIm1hbmFnZS1hY2NvdW50IiwibWFuYWdlLWFjY291bnQtbGlua3MiLCJ2aWV3LXByb2ZpbGUiXX1
9LCJzY29wZSI6ImVtYWlsIHByb2ZpbGUiLCJjbGllbnRJZCI6InNlcnZpY2VhY2NvdW50IiwiY2xpZW50SG9zdCI6IjEwL
jIwNS41LjEwMiIsImVtYWlsX3ZlcmlmaWVkIjpmYWxzZSwiYWNjb3VudF9leHRlcm5hbF9pZCI6Indmc19lbmdfdmFsaWR
fdXM4IiwiZmVhdHVyZV9mbGFncyI6WyJjbG9ja19jYXJkIiwiYXBpLWdldC1iYW5rIiwiY2xvY2tfY2FyZF9nZW9fbG9jY
XRpb24iLCJ1bWFfYXV0aG9yaXphdGlvbiIsImFwaS1nZXQtZW1wbG95ZWUtZ3JvdXAiLCJvZmZsaW5lX2FjY2VzcyIsImF
waS1nZXQtYmFuay1ldmVudHMiLCJhcGktZ2V0LWVtcGxveWVlIiwid3RhIl0sImFjY291bnRfaW50ZXJuYWxfaWQiOiJ3Z
nNfZW5nX3ZhbGlkX3VzOCIsInByZWZlcnJlZF91c2VybmFtZSI6InNlcnZpY2UtYWNjb3VudC1zZXJ2aWNlYWNjb3VudCI
sImNsaWVudEFkZHJlc3MiOiIxMC4yMDUuNS4xMDIiLCJyZWFsbV9uYW1lIjoid2ZzX2VuZ192YWxpZF91czgifQ.Iuo_xI
N1Wl16wFCm5fwYPxXxVv7CbyY-fe_KXkgdvazQKOI2m6Fa5-jKpn2lwvNqvXAR4-za-MDI2YNAc-
511xLJHo2wCuWgnLTVyTT-geeaXPDcINokKh7L83GO7Y-
4WxTPxTvFqhayytFaUIDVP_032Ua8tosF58u5iaBGgk8wdnl0qQut4K9i6ppj78cLr7z5TwUn-
WgTOcUcG41ZpTCGfkybx4s0hcfAXSTNfu1jquhO9CChRlgL5ndc3cED8WQ5VXrSctWB7vD2fsD-
TwblIeDPKRboEQ1z3arE78iNnMY6zSfDneImWN9CdQMwj7pcWbHLpCDMwAc5zb_RhQ","expires_in":300,"refresh_
expires_in":900,"refresh_token":"eyJhbGciOiJIUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJhODJhMDVlZ
C0wMzEzLTQ1MzYtYTFkNi05NGI0ZTRkNjZhMDcifQ.eyJleHAiOjE2ODQ0MTY3ODksImlhdCI6MTY4NDQxNTg4OSwianRp
IjoiZmZlZjVkMTAtZTRmZi00NmEzLThlMDYtNzQ3ZDI2MDE3OTFhIiwiaXNzIjoiaHR0cHM6Ly9jYXMtdXM4Lndmcy5jbG
91ZC9hdXRoL3JlYWxtcy93ZnNfZW5nX3ZhbGlkX3VzOCIsImF1ZCI6Imh0dHBzOi8vY2FzLXVzOC53ZnMuY2xvdWQvYXV0
aC9yZWFsbXMvd2ZzX2VuZ192YWxpZF91czgiLCJzdWIiOiIwMDBlYzJhOC0zNzQ4LTRjZWUtYjE4MC1kNDg2YTUxNWY1ND
IiLCJ0eXAiOiJSZWZyZXNoIiwiYXpwIjoic2VydmljZWFjY291bnQiLCJzZXNzaW9uX3N0YXRlIjoiNGVkNzBiZWYtM2I4
OC00NTFiLTk4OGEtYTFhOTg0ZGM4ZjkyIiwic2NvcGUiOiJlbWFpbCBwcm9maWxlIn0.ZX-
A3mPSgCxK4opZRioqqiqY3fRFg63wAh4kbW-4ogk","token_type":"bearer","not-before-
policy":0,"session_state":"4ed70bef-3b88-451b-988a-a1a984dc8f92","scope":"email profile"}
```


The `access_token` value should be used as your Authorization access token when calling the API.

The `expires_in` value tells you how long the `token` is valid. In this case, fifteen minutes.

How to Call a Data Feed API

Make a call to a Data Feed API like this:

```
GET
https://{{host.api}}/time/v1?count=1
```

The `host.api` will be provided to you securely by Customer Support.

As described above `host.api` is in the form `api-dc.wfs.cloud`, where dc is the data center (for example, US2, EU1, AU3, CA1, etc.).

The first time you call a Data Feed API it will probably be without a cursor, as you want the data from the beginning of the feed.

The value of count can be up to 10,000. If you do not provide a count the default value is 2,000. To authenticate this request, you must also include a header called `Authorization` which should be set using the `access_token` retrieved from the authentication service prepended with the word "Bearer" and a space, as described in Making a Request.

For example, using the command line tool `wget`, the GET request would look like this:

```
wget --no-check-certificate --quiet \
  --method GET \
  --timeout=0 \
  --header 'Authorization:
Bearer eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJNaVJHdkFxM3NsYkZWcHBmWkxyVlU4UlNaSk5
YS2plM1c3S0Nuc09obUFzIn0.eyJleHAiOjE1OTExOTcwMzAsImlhdCI6MTU5MTE5NTIzMCwianRpIjoiNmFlNzZiZTMtN
DdmMy00YzUzLWJjMzgtYTlhZDMwZmI4YmQ0IiwiaXNzIjoiaHR0cHM6Ly9jYXMtc3ByaW50LXVzNC51c2Rldi53ZnNhYXM
ubmV0L2F1dGgvcmVhbG1zL2dzZF9zX29uZSIsImF1ZCI6ImFjY291bnQiLCJzdWIiOiIyNzExYzMzNC0wOWMzLTRmZmYtO
GQ2Yi1jN2Y3OTQxNmNlNjIiLCJ0eXAiOiJCZWFyZXIiLCJhenAiOiJzZXJ2aWNlYWNjb3VudCIsInNlc3Npb25fc3RhdGU
iOiJhMjA0ZjA1NC0wNjFkLTQ2MTMtYTBmZC1lNWU4YWQ3MDg4NWYiLCJhY3IiOiIxIiwicmVhbG1fYWNjZXNzIjp7InJvb
GVzIjpbIm9mZmxpbmVfYWNjZXNzIiwidW1hX2F1dGhvcml6YXRpb24iXX0sInJlc291cmNlX2FjY2VzcyI6eyJhY2NvdW5
0Ijp7InJvbGVzIjpbIm1hbmFnZS1hY2NvdW50IiwibWFuYWdlLWFjY291bnQtbGlua3MiLCJ2aWV3LXByb2ZpbGUiXX19L
CJzY29wZSI6ImVtYWlsIHByb2ZpbGUiLCJjbGllbnRfcm9sZXMiOiJbb2ZmbGluZV9hY2Nlc3MsIHVtYV9hdXRob3JpemF
0aW9uXSIsImNsaWVudEhvc3QiOiIxMC4xOTIuNjQuNDQiLCJlbWFpbF92ZXJpZmllZCI6ZmFsc2UsImNsaWVudElkIjoic
2VydmljZWFjY291bnQiLCJhY2NvdW50X2V4dGVybmFsX2lkIjoiZ3NkX3Nfb25lIiwiYWNjb3VudF9pbnRlcm5hbF9pZCI
6ImdzZF9zMSIsInByZWZlcnJlZF91c2VybmFtZSI6InNlcnZpY2UtYWNjb3VudC1zZXJ2aWNlYWNjb3VudCIsImNsaWVud
EFkZHJlc3MiOiIxMC4xOTIuNjQuNDQiLCJlbWFpbCI6InNlcnZpY2UtYWNjb3VudC1zZXJ2aWNlYWNjb3VudEBwbGFjZWh
vbGRlci5vcmciLCJyZWFsbV9uYW1lIjoiZ3NkX3Nfb25lIn0.VLFWNFom5Dc4EcIST5eIXH62i0Zw9uJB30ZyUB2CI5omH
wgO7zX6ZA1X_ju7agrdmQIGuqAiwXScX9E2DGwahRFqkj_JN-
YT2FfssUip_eApo3wBSLVVO4I2jKvKXs9Nne6mgTpK5WpyEa4ff_aI55eFSZaw-YGSCL5U8buser-
5R_TdejmIas8EazlAG9G15fXbowzwujKo-2JddKkkzLM3eOS7Ww-
TezlYYCWObQoitFFNkaynSbkZD2A3kvEOi5kz8IvnPDB74vjBTk0U34G63DGgFfyMcWaT0EjY6ErVORUluT89BXLxGDx18
Oq2tTcBhiNNorcvTa7JZyfBYF55Mg' \
   'https://{{host.api}}/time/v1?count=1'
```

The response to this call will be like the following:


```
{"updateSequence":[{
  "recordId" : "Z3NkX3MyLTE5NTk5ODU5MTY=",
  "dataChange" : "MERGE",
  "externalMatchId" : "USA03-009",
  "startAt" : "2019-03-24",
  "endAt" : "2019-03-30",
  "times" : null,
  "exceptions" : null
}],
"nextCursor":"%23dNcVuTeMteq98%2FfiNreGcfmyk%2Fqjx0P1SJt0AyCHVoT4HHJHdPy6fH0eBYmBa2jw
NJ4Gekg6h9UMOMvZrM2fBW3%2FM%2FodSt7mawfhhE6mjDrS5YT%2BmXc9MLFy7ZMj3%2BDoauzTDp%2BqZoI
IPCOEAQ60dA%3D%3D"}
```


The `updateSequence` data is a sequence of records. In this case, a single Time record was received because the count was set to 1. Details of the data formats for each API can be found in the OpenAPI spec referenced at docs.integration.wfs.cloud.

The `recordId` value is a unique identifier for this record. Any future updates to this record will appear in the feed with the same `recordId`. This allows clients to avoid duplication in their datastore by only keeping the latest record if that is the desired behavior.

The `dataChange` value indicates whether this record is to be deleted or whether the record is new or an update. Records where `dataMerge` is set to DELETE are no longer operational in the WFS system and can be deleted. Those marked as MERGE are either new or an update to a record which has already appeared in the feed. Records to be updated will match against a previous record's `recordId`.

The `nextCursor` value is used to retrieve data from the next point in the feed by passing this as a parameter in the next call.

An example of using `nextCursor`:

```
GET
https://{{host.api}}{{time.port}}/time/v1?count=1&cursor=%23dNcVuTeMteq98%2FfiNreGcfm
yk%2Fqjx0P1SJt0AyCHVoT4HHJHdPy6fH0eBYmBa2jwNJ4Gekg6h9UMOMvZrM2fBW3%2FM%2FodSt7mawfhhE
6mjDrS5YT%2BmXc9MLFy7ZMj3%2BDoauzTDp%2BqZoIIPCOEAQ60dA%3D%3D
```

How to Call Standard Application APIs

Other WorkForce APIs which are non-data feed style, allow you to send data to WorkForce to update the system with new information (e.g., a clock swipe or a new hire) or to query the system (e.g., Timesheet Summary API). These APIs are intended to provide end points for system-to-system integration (e.g., integrating an HR system with the WorkForce Platform). Typically, this is achieved using the customer's own integration platform.

To use these APIs, you need to authenticate in the same way as described for data feeds above.

The following is an example of how to send a clock event to WorkForce:

```
PUT
https://{{host.api}}/clock/v1
```

You must add the `Authorization` header for authentication containing a valid token, as described above.

The body of the message will be like the following:

```
{
  "displayEmployee": "Test_15018_01",
  "eventId": "TESTSwipe0056381",
  "timestamp": "2020-03-20T08:00:00.000-04:00",
  "payCode": "REG",
  "clockId": "iains-postman",
  "eventType": "SWIPE_IN"
}
```

Note: *This message body is an example. Please check the documentation for details of message formats.*

This PUT request will receive a response like this:

```
{
  "timestamp" : "2020-06-04T05:49:18.194-04:00",
  "status" : "200",
  "path" : "/clock/v1",
  "eventId" : " TESTSwipe0056381"
}
```

Errors in the requests will result in an error message in the response. For example, omitting the trailing "}" in the body will result in this response.

```
{
    "timestamp": "2020-06-04T09:51:24.427+0000",
    "status": 400,
    "error": "Bad Request",
    "message": "Unexpected end-of-input: expected close marker for Object (start
marker at [Source: (String)\"{\r\n  \"displayEmployee\": \"Test_15018_01\",\r\n
\"eventId\": \"TESTSwipe00001\",\r\n  \"timestamp\": \"2020-03-20T08:00:00.000-
04:00\",\r\n  \"payCode\": \"REG\",\r\n  \"clockId\": \"iains-postman\",\r\n
\"eventType\": \"SWIPE_IN\"\r\n\"; line: 1, column: 1])\n at [Source: (String)\"{\r\n
\"displayEmployee\": \"Test_15018_01\",\r\n  \"eventId\": \"TESTSwipe00001\",\r\n
\"timestamp\": \"2020-03-20T08:00:00.000-04:00\",\r\n  \"payCode\": \"REG\",\r\n
\"clockId\": \"iains-postman\",\r\n  \"eventType\": \"SWIPE_IN\"\r\n\"; line: 8,
column: 203]",
    "path": "/clock/v1"
}
```

When asynchronous processing is involved, errors will be reported via email.

REST API Endpoints

Bank

GET /bank/v1

Description

The Bank API will retrieve the balance of the banks as updates are made over time. The balance will update at least once per period. This is a data feed API.

Bank balances are defined as the employee balances coming from source systems—i.e., holiday, time off, sick days. The source system may use arbitrary non-overlapping date ranges for the messages; those date ranges must be stable when re-publishing data—i.e., they are not tied to something fixed like pay periods but are not expected to change going forward.

For an example response and a more detailed view of the REST API schemas, see Banks API OpenAPI Specification.

Bank Events

GET /bank-events/v1

Description

The Bank Event Data Feed API retrieves a history of bank accruals and usage events from any and all sources. For example, an event would be when the vacation bank is decremented when vacation is placed on the employee's timesheet. This API does not retrieve a bank balance history.

For an example response and a more detailed view of the REST API schemas, see Bank Events OpenAPI Specification.

Calculated Time

GET /calculated-time/v1

Description

The Calculated Time Data Feed API retrieves employee time records after they have been calculated for employee pay. For example, time worked, breaks, and vacation time are calculated to provide the time records used in the employee's pay. These records are updated every time a timesheet is calculated.

For an example response and a more detailed view of the REST API schemas, see Calc Time OpenAPI Specification.

GET /calculated-time/employee/v1

Description

The Employee Time Data Feed API retrieves employee(s) time records for specified date range after they have been calculated for employee pay.

For example, time worked, breaks, and vacation time are calculated to provide the time records used in the employee's pay. These records are updated every time a timesheet is calculated.

For an example response and a more detailed view of the REST API schemas, see Calc Time Employee OpenAPI Specification.

Clock

GET /clock/v1

The clock/v1 endpoint has been deprecated and replaced by GET /swipe/v1.

Clock

PUT /clock/v1

Description

Create a clock event for an employee. This API is for use by non-WorkForce Software clocks, clock managers, or other data collection devices.

For an example response and a more detailed view of the REST API schemas, see Clock OpenAPI Specification.

Data Translation

GET /data-translation/v1

Description

The Data Translation API retrieves a record of the language translations that have been made for a given locale ID. This is a data feed API.

For an example response and a more detailed view of the REST API schemas, see Data Translation OpenAPI Specification.

Employee

GET /employee/v1

Description

The Employee Data Feed API retrieves updates to employee and job records as they are made. Job records are also commonly referred to as WT&A assignment records.

For an example response and a more detailed view of the REST API schemas, go to Employee OpenAPI Specification.

Employee Group

GET /employee-group/v1

Description

The Employee Group Data Feed API return WT&A assignment groups in addition to groups published by other services.

For an example response and a more detailed view of the REST API schemas, see Employee Group OpenAPI Specification.

Errors and Warnings

GET /errors-and-warnings/v1

Description

The Errors and Warnings Data Feed API retrieves all errors and warnings for the Person API, Time API and HCM Connectors, including CSV Connector.

For an example response and a more detailed view of the REST API schemas, see Errors & Warnings OpenAPI Specification.

Pay

GET /pay/v1

Description

The Pay Data Feed API retrieves Calc Pay records. Unlike Calc Time, which is published every time the timesheet is re-calculated and reflects just the latest time for a given date, Calc Pay is only published at the point the timesheet is locked and reflects the entire data set that is considered "to be paid" (including any associated amendments) for the period as a whole.

For an example response and a more detailed view of the REST API schemas, see Pay OpenAPI Specification.

Person

GET /person/v2

Description

The Person API retrieves all data for a person by querying on one of – externalMatchId, displayId or loginId.

For an example response, and a more detailed view of the REST API schemas, see Person OpenAPI Specification.

PUT /person/v2

Description

Create or update a person, uniquely identified by 'externalMatchId' in the request body.

See Appendix A: Field Mappings for the Person API v2 for more information on the format/fields required for the Person API.

For an example response and a more detailed view of the REST API schemas, see Person OpenAPI Specification.

How To Use this API

New Hire

To hire a new employee with an externalMatchId of A91378 and a hire date of 2020-09-21, and with a single job (also starting on the same date), call the Person API with the following:

New hire with just employee

```
{
    "externalMatchId": "A91378",
    "name": {
        "displayName": "Jonie Doe"
    },
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": true,
        "originalHireDate": "2020-09-21",
        "effectiveDatedEmployeeInfo": [
```

```
            {
                "effectiveDate": "2020-09-21",
                "latestHireDate": "2020-09-21"
            }
        ],
        "jobs": [
            {
                "externalJobId": "MI-LIV-4536",
                "isFullJobHistory": true,
                "jobDescription": "Michigan Livonia Driver",
                "originalStartDate": "2020-09-21",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-21",
                        "isActive": true,
                        "latestStartDate": "2020-09-21",
                        "timezone": "America/Detroit"
                    }
                ]
            }
        ]
    }
}
```

Since most employees should also be users, you can create both an employee and a user for that employee at the same time in a single request. To hire a new employee with an externalMatchId of A91378 and a hire date of 2020-09-01, with a single job (also starting on the same date) and with an accompanying user record with loginId of jdoe, call the Person API with the following:

New hire with both employee and user

```
{
    "externalMatchId": "A91378",
    "name": {
        "displayName": "Jonie Doe"
    },
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": true,
        "originalHireDate": "2020-09-21",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-21",
                "latestHireDate": "2020-09-21"
            }
        ],
        "jobs": [
            {
```

```
                "externalJobId": "MI-LIV-4536",
                "isFullJobHistory": true,
                "jobDescription": "Michigan Livonia Driver",
                "originalStartDate": "2020-09-21",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-21",
                        "isActive": true,
                        "latestStartDate": "2020-09-21",
                        "timezone": "America/Detroit"
                    }
                ]
            }
        ]
    },
    "user": {
        "loginId": "jdoe",
        "enabled": true
    }
}
```

Terminate an employee

An employee is considered terminated when they no longer have any active jobs. However, in addition to the job-level information the employee-level information also tracks a termination date. This allows for easy calculations to be made based on the date of the employee's termination, instead of requiring information from all of the employee's jobs to be evaluated. To terminate an employee with externalMatchId of A91378, where the last date that the employee was working is 2020-12-08, call the Person API with the following:

Employee termination

```
{
    "externalMatchId": "A91378",
    "name": {
        "displayName": "Jonie Doe"
    },
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": true,
        "originalHireDate": "2020-09-21",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-21",
                "latestHireDate": "2020-09-21"
            },
            {
                "effectiveDate": "2020-12-08",
                "latestHireDate": "2020-09-21",
                "terminationDate": "2020-12-08"
```

```
            }
        ],
        "jobs": [
            {
                "externalJobId": "MI-LIV-4536",
                "isFullJobHistory": true,
                "jobDescription": "Michigan Livonia Driver",
                "originalStartDate": "2020-09-21",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-21",
                        "isActive": true,
                        "latestStartDate": "2020-09-21",
                        "timezone": "America/Detroit"
                    },
                    {
                        "effectiveDate": "2020-12-08",
                        "isActive": false,
                        "latestStartDate": "2020-09-21",
                        "endDate": "2020-12-08",
                        "timezone": "America/Detroit"
                    }
                ]
            }
        ]
    },
    "user": {
        "loginId": "jdoe",
        "enabled": false
    }
}
```


The terminationDate is expected to represent the last active date for the employee, and not the first inactive date for the employee.

The user record associated with the employee should also typically be disabled when the employee is terminated. (This is accomplished by setting the enabled field in the user section of the request to false.) However, if the user should continue to be able to log in and perform other actions, such as approving other employees' timesheets, then it is possible to leave the user record active even for a terminated employee.

Re-hire

When a terminated employee is rehired, their original employee record can be reactivated again in the WorkForce Suite by sending over an additional effective-dated record that restores at least one active job for the employee. At this point, it is expected that the latestHireDate for the employee and the latestStartDate for the job will be updated to reflect the date of the rehire. (On that rehire’s effective-dated record and later records only. The effective-dated

records corresponding to the original hire period are expected to retain the original values for those fields that they had previously held. Additionally, the originalHireDate at the employee level and the originalStartDate at the job level are not expected to change values.) To re-hire an employee (and re-enable as a user) with externalMatchId of A91378, with a re-hire date of 2021-06-13, call the Person API with the following:

Rehiring an employee

```
{
    "externalMatchId": "A91378",
    "name": {
        "displayName": "Jonie Doe"
    },
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": true,
        "originalHireDate": "2020-09-21",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-21",
                "latestHireDate": "2020-09-21"
            },
            {
                "effectiveDate": "2020-12-08",
                "latestHireDate": "2020-09-21",
                "terminationDate": "2020-12-08"
            },
            {
                "effectiveDate": "2021-06-13",
                "latestHireDate": "2021-06-13"
            }
        ],
        "jobs": [
            {
                "externalJobId": "MI-LIV-4536",
                "isFullJobHistory": true,
                "jobDescription": "Michigan Livonia Driver",
                "originalStartDate": "2020-09-21",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-21",
                        "isActive": true,
                        "latestStartDate": "2020-09-21",
                        "timezone": "America/Detroit"
                    },
                    {
                        "effectiveDate": "2020-12-08",
                        "isActive": false,
                        "latestStartDate": "2020-09-21",
                        "endDate": "2020-12-08",
                        "timezone": "America/Detroit"
```

```
                    },
                    {
                        "effectiveDate": "2021-06-13",
                        "isActive": true,
                        "latestStartDate": "2021-06-13",
                        "timezone": "America/Detroit"
                    }
                ]
            }
        ]
    },
    "user": {
        "loginId": "jdoe",
        "enabled": true
    }
}
```

Create a new user

Users can also be created, separate from employees, to allow people who do not need to track any of their own time/scheduling to perform actions against other employees (such as approving timesheets or time off requests). To create a new user with an externalMatchId of JD123, an ssoMatchId of jd123-a and an email address of jd123@workforcesoftware.com, call the Person API with the following (note that invalid timezones will error):

Creating a new user

```
{
    "externalMatchId": "JD123",
    "emailAddress": "jd123@workforcesoftware.com",
    "name": {
        "firstName": "John",
        "lastName": "Doe",
        "displayName": "John Doe"
    },
    "user": {
        "loginId": "JD123",
        "timezone": "America/Detroit",
        "enabled": true,
        "ssoMatchId": "jd123-a"
    }
}
```

Disable a user

Users are not terminated, instead they are disabled. This change is effective immediately (user data is not effective dated) and prevents the user from logging in or performing any actions in

the system for themselves or others. For example, to disable a user with an externalMatchId of A91378, call the Person API with the following:

Disabling a user

```
{
    "externalMatchId": "A91378",
    "user": {
        "loginId": "jdoe",
        "enabled": false
    }
}
```

Terminate a job

If an employee has more than one job, one of their jobs can be terminated without terminating the employee. In this case, the timesheets/schedules associated with the terminated job will no longer be tracked after the termination date, but the timesheets/schedules associated with the other remaining active jobs will continue to be processed normally. Assume that the following data has previously been loaded for the employee, indicating that they currently have two active jobs with IDs MI-LIV-4536 and MI-LIV-9120:

Initial employee data

```
{
    "externalMatchId": "A91378",
    "name": {
        "displayName": "Jonie Doe"
    },
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": true,
        "originalHireDate": "2020-09-21",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-21",
                "latestHireDate": "2020-09-21"
            }
        ],
        "jobs": [
            {
                "externalJobId": "MI-LIV-4536",
                "isFullJobHistory": true,
                "jobDescription": "Michigan Livonia Driver",
                "originalStartDate": "2020-09-21",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-21",
```

```
                        "isActive": true,
                        "latestStartDate": "2020-09-21",
                        "timezone": "America/Detroit"
                    }
                ]
            },
            {
                "externalJobId": "MI-LIV-9120",
                "isFullJobHistory": true,
                "jobDescription": "Michigan Livonia Cafeteria Worker",
                "originalStartDate": "2021-01-04",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2021-01-04",
                        "isActive": true,
                        "latestStartDate": "2021-01-04",
                        "timezone": "America/Detroit"
                    }
                ]
            }
        ]
    }
}
```

To indicate that the job with ID MI-LIV-4536 should be terminated as of 2021-01-26, while the job with ID MI-LIV-9120 should remain active, the Person API should be called as follows:

Terminating a job

```
{
    "externalMatchId": "A91378",
    "name": {
        "displayName": "Jonie Doe"
    },
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": true,
        "originalHireDate": "2020-09-21",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-21",
                "latestHireDate": "2020-09-21"
            }
        ],
        "jobs": [
            {
                "externalJobId": "MI-LIV-4536",
                "isFullJobHistory": true,
                "jobDescription": "Michigan Livonia Driver",
                "originalStartDate": "2020-09-21",
```

```
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-21",
                        "isActive": true,
                        "latestStartDate": "2020-09-21",
                        "timezone": "America/Detroit"
                    },
                    {
                        "effectiveDate": "2021-01-26",
                        "isActive": false,
                        "latestStartDate": "2020-09-21",
                        "endDate": "2021-01-25",
                        "timezone": "America/Detroit"
                    }
                ]
            }
        ]
    }
}
```

Note that the call to the Person API to terminate just a single job can include information for the other jobs that should remain active or it cannot, at the caller’s discretion. Any jobs that are not included in the request that is made to the Person API will not be modified when the request is being processed, and their existing data will remain unchanged.

Change job/assignment

Assume that the following data was what was sent over for an employee initially, reflecting two effective-dated records for the employee's job reflecting a change from an initial job level of 1 to a job level of 2 on 2021-01-21:

Initial job data

```
Initial job data
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "jobs": [
            {
                "externalJobId": "MI-LIV_4536",
                "jobDescription": "Michigan Livonia Driver",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-04",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "1",
```

```
                        "locationId": "LOC123"
                    },
                    {
                        "effectiveDate": "2021-01-21",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "2",
                        "locationId": "LOC123"
                    }
                ]
            }
        ]
    }
}
```

In order to modify the data for that job to completely replace the information that was previously loaded with the new data, just send the data again with the same effective date and the new values. This will cause the existing effective-dated information for that effective date to be replaced with the new information, making that new data effective for that effective-date range. For instance, the following example will change the job's location from LOC123 to LOC456 for the effective-dated range starting on 2020-09-04 while preserving the original location on the effective-dated range starting on 2021-01-21:

Job with in-place effective-dated change

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "jobs": [
            {
                "externalJobId": "MI-LIV_4536",
                "jobDescription": "Michigan Livonia Driver",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-04",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "1",
                        "locationId": "LOC456"
                    },
                    {
                        "effectiveDate": "2021-01-21",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "2",
                        "locationId": "LOC123"
                    }
                ]
```

```
            }
        ]
    }
}
```

Note that in the above example, the 2021-01-21 record still needs to be included even though it contains no changes. This is because if it is not included, the import process will view the record effective on 2020-09-04 not as running from 2020-09-04 – 2021-01-20, as it was originally imported, but instead as running from 2020-09-04 through the end of time. This would effectively eliminate the second record that was originally present.

In order to add new changes to the job, while preserving the original data intact, there are two approaches that can be used. The first would be to send the entire set of data as it should now exist for the job. For instance, the following example demonstrates how to send a change to move the job from location LOC123 to location LOC456 starting on 2021-02-01:

Job change with additional new effective-dated record

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "jobs": [
            {
                "externalJobId": "MI-LIV_4536",
                "jobDescription": "Michigan Livonia Driver",
                "isFullJobHistory": true,
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-04",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "1",
                        "locationId": "LOC123"
                    },
                    {
                        "effectiveDate": "2021-01-21",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "2",
                        "locationId": "LOC123"
                    },
                    {
                        "effectiveDate": "2021-02-01",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "2",
                        "locationId": "LOC456"
                    }
```

```
                ]
            }
        ]
    }
}
```

An alternative approach that could also be used would be to just send over the newly-changed data, using the partial history mode. The following example demonstrates how to send just the 2021-02-01 data from the previous example, without needing to include the rest of the data in the stack. This minimizes the amount of data that needs to be sent to the Person API, but increases the risk that data could get out of sync between the source HR system and the WorkForce Suite.

Job change with partial-history effective-dated change

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "jobs": [
            {
                "externalJobId": "MI-LIV_4536",
                "jobDescription": "Michigan Livonia Driver",
                "isFullJobHistory": false,
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2021-02-01",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "jobLevel": "2",
                        "locationId": "LOC456"
                    }
                ]
            }
        ]
    }
}
```

Update an employee

Assume that the following data was what was sent over for an employee initially, reflecting two effective-dated records for the employee reflecting a change from a badge ID of 12345 to a badge ID of 67890 on 2021-01-21:

Initial employee data

```
{
    "externalMatchId": "A91378",
```

```
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "importedBadgeId": "12345"
            },
            {
                "effectiveDate": "2021-01-21",
                "importedBadgeId": "67890"
            }
        ]
    }
}
```

In order to modify the data for that employee to completely replace the information that was previously loaded with the new data, just send the data again with the same effective date and the new values. This will cause the existing effective-dated information for that effective date to be replaced with the new information, making that new data effective for that effective-date range. For instance, the following example will change the employee's badge ID from 12345 to 54321 for the effective-dated range starting on 2020-09-04 while preserving the badge ID of 67890 on the effective-dated range starting on 2021-01-21:

Employee with in-place effective-dated change

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "importedBadgeId": "54321"
            },
            {
                "effectiveDate": "2021-01-21",
                "importedBadgeId": "67890"
            }
        ]
    }
}
```

Note that in the above example, the 2021-01-21 record still needs to be included even though it contains no changes. This is because if it is not included, the import process will view the record effective on 2020-09-04 not as running from 2020-09-04 – 2021-01-20, as it was

originally imported, but instead as running from 2020-09-04 through the end of time. This would effectively eliminate the second record that was originally present.

In order to add new changes to the employee, while preserving the original data intact, there are two approaches that can be used. The first would be to send the entire set of data as it should now exist for the employee. For instance, the following example demonstrates how to send a change to move the employee from badge ID 67890 to badge ID 54321 starting on 2021-02-01:

Employee change with additional new effective-dated record

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": true,
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "importedBadgeId": "54321"
            },
            {
                "effectiveDate": "2021-01-21",
                "importedBadgeId": "67890"
            },
            {
                "effectiveDate": "2021-02-01",
                "importedBadgeId": "54321"
            }
        ]
    }
}
```

An alternative approach that could also be used would be to just send over the newly-changed data, using the partial history mode. The following example demonstrates how to send just the 2021-02-01 data from the previous example, without needing to include the rest of the data in the stack. This minimizes the amount of data that needs to be sent to the Person API, but increases the risk that data could get out of sync between the source HR system and the WorkForce Suite.

Employee change with partial-history effective-dated change

```
Employee change with partial-history effective-dated change
{
    "externalMatchId": "A91378",
    "employee": {
```

```
        "displayId": "A91378",
        "isFullEmployeeHistory": false,
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2021-02-01",
                "importedBadgeId": "54321"
            }
        ]
    }
}
```

Update a user

Assume that the following data was what was sent over for a userinitially, reflecting a user that does not have a business objects user ID:

Initial user data

```
{
    "externalMatchId": "A91378",
    "user": {
        "loginId": "A91378",
        "enabled": true,
        "businessObjectsId": ""
    }
}
```

In order to modify the user to have a business objects user ID the same user data should be sent again, this time with the updated values:

Updated user data

```
{
    "externalMatchId": "A91378",
    "user": {
        "loginId": "A91378",
        "enabled": true,
        "businessObjectsId": "BO1"
    }
}
```

It is important to note that, unlike employee and job information, user data is not effective dated. This means that all of the data that should exist for the user needs to be sent every time the user is updated.

Adding someone as a manager of a division in scheduling

The following example shows how to specify that a user is the manager of a division within WF&S. The user in this example is the manager of the divisions DivA, DivB, and DivC

Specifying the divisions managed by a user

```
{
    "externalMatchId": "A91378",
    "user": {
        "loginId": "A91378",
        "enabled": true,
        "managedSchedulingUnits": [
            "DivA",
            "DivB",
            "DivC"
        ]
    }
}
```

Adding a person as someone's manager

The manager relationship is defined by two fields:

The first is the managersMatchValue, which is defined as an attribute of the Job data. This value reflects which managers are the managers for that job:

Specifying the manager for a job

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "jobs": [
            {
                "externalJobId": "MI-LIV_4536",
                "jobDescription": "Michigan Livonia Driver",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-04",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "managersMatchValue": "ABC123"
                    }
                ]
            }
        ]
    }
}
```

The second is the managedJobsMatchValue, which is defined as an attribute of the employee data. This value reflects which jobs this employee manages:

Specifying the jobs managed by an employee

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "managerIndicator": true,
                "managedJobsMatchValue": "ABC123"
            }
        ]
    }
}
```

One important note is that, in order for the manager relationship to be defined, the only characteristic of the data that matters is that the managedJobsMatchValue specified in the employee data for the manager matches the managersMatchValue specific in the job data for the jobs that they manager. This value can be anything, so long as it is the same in both cases. Some common examples are as follows:

A job reports directly to a single, specific manager

If a given job has one (and only one) manager that it reports to, typically the match value that is used would be the manager's employee ID. This means that the data would generally look like the following for the manager:

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "managerIndicator": true,
                "managedJobsMatchValue": "A91378"
            }
        ]
    }
}
```

And the data would generally look like the following for the job of the person who reports to that manager:

```
{
    "externalMatchId": "A12345",
    "employee": {
        "displayId": "A12345",
        "jobs": [
            {
                "externalJobId": "MI-LIV_4536",
                "jobDescription": "Michigan Livonia Driver",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-04",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "managersMatchValue": "A91378"
                    }
                ]
            }
        ]
    }
}
```

In this case, the manager's managedJobsMatchValue is their own employee ID, and the job for the employee(s) they manage has a managersMatchValue that is also that manager's employee ID. This way, we've indicated that manager A91378 manages all jobs that report to "A91378", and indicated that the job for employee A12345 reports to manager A91378.

Any number of managers may manage all jobs for a given location

If there are any number of managers that manage all jobs with a specific attribute—such as working in the same location—then the match value that is used would be the value of that attribute. For example, in the scenario where all employees in location "Warehouse" are managed by several different managers, the data for the managers would look like:

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "managerIndicator": true,
                "managedJobsMatchValue": "Warehouse"
            }
        ]
    }
}
```

Or:

```
{
    "externalMatchId": "A98765",
    "employee": {
        "displayId": "A98765",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "managerIndicator": true,
                "managedJobsMatchValue": "Warehouse"
            }
        ]
    }
}
```

Note that in this scenario, multiple managers can have the same managedJobsMatchValue.

The data for the jobs that report to these managers would look like:

```
{
    "externalMatchId": "A12345",
    "employee": {
        "displayId": "A12345",
        "jobs": [
            {
                "externalJobId": "MI-LIV_4536",
                "jobDescription": "Michigan Livonia Driver",
                "effectiveDatedJobInfo": [
                    {
                        "effectiveDate": "2020-09-04",
                        "isActive": true,
                        "timeZone": "America/Detroit",
                        "managersMatchValue": "Warehouse"
                    }
                ]
            }
        ]
    }
}
```

In this scenario, all of the managers for the location have the same managedJobsMatchValue, which is the ID for the location. All of the jobs working in that location also have that location's ID as their managersMatchValue. This way, we've indicated that employees A91378 and A98765 both manage all jobs that work in the Warehouse, and that the job for employee A12345 is a job that works in the Warehouse.

Promotion/demotion

Typically for an employee that has been promoted to be a manager, there are a couple of changes that are expected to happen to their data. Assume that prior to this promotion, the employee has the following data:

Initial employee data

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2020-09-04",
                "managerIndicator": false
            }
        ]
    }
}
```

In order to make them a manager, we would expect to see a new effective-dated record supplied for them that sets the managerIndicator value to true and to see a managedJobsMatchValue that specifies which jobs they now manage. This could look like the following:

Data to change employee into a manager

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2021-01-21",
                "managerIndicator": true,
                "managedJobsMatchValue": "A91378"
            }
        ]
    }
}
```

For details on the specific values expected for the managedJobsMatchValue, please see the Adding a person as someone's manager section above.

If a manager is no longer a manager, the same sort of update can be applied in reverse to make the employee cease to be a manager. Specifically, we would expect to see a new effective-dated record supplied for them that sets the managerIndicator back to false and clears out the managedJobsMatchValue. This could look like the following:

Data to change employee to no longer be a manager

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2021-02-01",
                "managerIndicator": false
            }
        ]
    }
}
```

Partial history

Partial history processing is the default processing mode of the Standard Import using the Person API. To process a change in partial historic mode, call the Person API with a request that provides a value of false for either the isFullEmployeeHistory and/or the isFullJobHistory properties:

Partial history at the employee level

```
{
    "externalMatchId": "A91378",
    "employee": {
        "displayId": "A91378",
        "isFullEmployeeHistory": false,
        "effectiveDatedEmployeeInfo": [
            {
                "effectiveDate": "2021-02-01",
                "managerIndicator": false
            }
        ]
    }
}
```

Partial history at the job level

```
{
  "externalMatchId": "A91378",
    "employee": {
      "displayId": "A91378",
```

```
      "jobs": [
        {
        "externalJobId": "MI-LIV-4536",
        "isFullJobHistory": false,
        "jobDescription": "Michigan Livonia Driver",
        "effectiveDatedJobInfo": [
          {
          "effectiveDate": "2020-09-04",
          "isActive": false,
          "endDate": "2020-09-18"
          }
        ]
      }
    ]
  }

}
```

If the value provided instead is true, the import will treat the supplied effective dates as representing the full history that should exist and will remove any records from the existing history that are effective before the earliest supplied effective date.

Policy Set

GET /policy-set/v1

This API has been deprecated. Please contact WorkForce Software for alternatives.

Description

The Policy Set Data Feed API retrieves a record of the policy sets that have been published by WT&A.

For an example response and a more detailed view of the REST API schemas, go to Policy Set OpenAPI Specification.

Role Detail

GET /role-detail/v1

Description

The Role Detail Data Feed API retrieves a record of the system features associated with user roles from all WorkForce Suite services. This API can be used to monitor changes to roles within the WorkForce Suite.

For an example response and a more detailed view of the REST API schemas, go to Role Detail OpenAPI Specification.

Shift

GET /shift/v1

Description

The Shift Data Feed API retrieves in/out schedule data. If Classic Standard Scheduling or Classic Job Scheduling is in use, then the data source will be WT&A. If any other scheduling product is in use, then that scheduling product is the data source.

For an example response and a more detailed view of the REST API schemas, go to Shift OpenAPI Specification.

GET /shift/employee/v1

Description

The Shift Queryable Data Feed API retrieves in/out schedule data for a specified employee within a given date range. If Classic Standard Scheduling or Classic Job Scheduling is in use, then the data source will be WT&A. If any other scheduling product is in use, then that scheduling product is the data source.

For an example response and a more detailed view of the REST API schemas, go to Shift OpenAPI Specification.

Swipe

GET /swipe/v1

Description

This endpoint replaces the now deprecated GET clock/v1.

The Swipe Data Feed API retrieves a record of processed clock swipes that have been collected from WorkForce Software compatible devices. This does not include manual adjustments. This API replaces the now deprecated GET clock/v1 endpoint.

For an example response and a more detailed view of the REST API schemas, go to Swipe OpenAPI Specification.

Time

GET /time/v1

Description

This endpoint has been deprecated. Please use the time-exceptions endpoint to retrieve time exception data. See Time Exceptions Open API Specification.

The Time Data Feed API retrieves exception data from time records. This API does not retrieve more general time data.

For an example response and a more detailed view of the REST API schemas, go to Time OpenAPI Specification.

PUT /time/v2

Description

Create or update time data for a single employee, identified by 'externalMatchId' in the request body. The actual processing of this information will be done by a Time Entry Import policy in WT&A, as specified on the Suite Import Settings policy in the WT&A configuration. As Time Entry Import policies are highly configurable and can have vastly different functionality depending on how they are configured, please see the WT&A Policy Configuration Guide section on configuring Time Entry Imports for more information on how to set up those policies and on what behaviors are possible when processing Time Requests.

For an example response and a more detailed view of the REST API schemas, go to Time OpenAPI Specification.

Time Exceptions

GET /time-exceptions/v1

Description

The Time Exceptions Data Feed API retrieves exception data from time records.

For an example response and a more detailed view of the REST API schemas, go to Time Exceptions OpenAPI Specification.

Time Off

GET /time-off/v1

Description

The Time Off Data Feed API retrieves all approved timer off requests.

For an example response and a more detailed view of the REST API schemas, go to Time Off OpenAPI Specification.

GET /time-off/employee/v1

Description

The Time Off Queryable Data Feed API retrieves all the time off requests for a specified employee within a given date range.

For an example response and a more detailed view of the REST API schemas, go to Time Off OpenAPI Specification.

PUT /time-off/v1

Description

The Time Off PUT API to create or update Approved time-off data for a single employee, identified by 'externalMatchId' in the request. This API can create or update time off requests, but currently, the PUT method is only available for Demand Scheduling customers. It is not available to customers whose Absence Management is handled by Time & Attendance.

For an example response and a more detailed view of the REST API schemas, go to Time Off OpenAPI Specification.

Time Off Request

GET /time-off-request/v1

Description

The Time Off Request Data Feed API retrieves all time off requests regardless of the current status of the request.

For an example response and a more detailed view of the REST API schemas, go to Time Off Request OpenAPI Specification.

Timesheet Summary

GET /timesheet-summary/v1

Description

The Timesheet Summary Data Feed API retrieves raw uncalculated time data. This includes time values as they are entered or imported. This request collects data 3 days back from the current date.

For an example response and a more detailed view of the REST API schemas, go to TimesheetSummary OpenAPI Specification.

User

GET /user/v1

Description

The User Data Feed API retrieves user record updates as they are made. Any change in the user record results in a new record in the API response.

For an example response and a more detailed view of the REST API schemas, go to User OpenAPI Specification.

User Rights

GET /user-right/v2

Description

Data Feed to return composite user rights of both WF&S and WT&A source systems in a single record.

For an example response and a more detailed view of the REST API schemas, go to User Right OpenAPI Specification.

Create a Data Store

Overview

The WorkForce data feed APIs are designed to give near real-time access to your data. These data feeds utilize a cursor concept allowing clients to pull data in the order in which it was updated or deleted in WorkForce, keeping track of where you are in the stream of events using the cursor. A call to the API with no *cursor* parameter being passed will result in the earliest record updates being returned. e.g., the earliest update to an employee. The number of records returned is controlled by the *count* parameter, 2000 records by default, with a maximum of 10,000 records.

The APIs have been kept as simple as possible, such that in most cases the data from the API can be stored in at most 2 tables e.g., a main domain object table for attributes which are not date effective and a linked table for those that are. Where data duplication is not a concern or a convenient reporting feature, data can often be stored in a single table if that is preferred but will require an appropriate composite primary key (PK) to be selected. The model described below tries to strike a balance, in most cases avoiding duplication. Note that there are objects in the model where no PK is marked. This happens where it is assumed data will always be queried as part of a join with a table which does have a PK. The model is only a recommended starting point and is open to extension and modification to suit different customer use cases. For example, there are many fields in Employee that will not be used by many customers, and these can obviously be omitted.

Before You Get Started

When planning to use APIs for reporting in a data warehouse environment, several key requirements need to be considered to ensure effective integration and reliable reporting. These requirements span various aspects of data management, integration, security, and scalability. Here's a breakdown of some essential data warehouse requirements:

1. Data Model Compatibility: Ensure that the data model of the data warehouse is compatible with the data structure provided by the APIs. This includes understanding the format, schema, and data types of the data exposed by the APIs and mapping them to the data warehouse schema.

2. API Access and Documentation: Access to the APIs should be well-documented, including details about authentication mechanisms, endpoint URLs, request parameters, response formats, rate limits, and any data transformation requirements. Clear documentation is crucial for developers to effectively integrate APIs with the data warehouse. Hopefully you will find all that in this guide.

3. Data Quality and Consistency: Implement mechanisms to ensure data quality and consistency when integrating data from APIs into the data warehouse. This may involve data validation, cleansing, deduplication, and error handling procedures to maintain the integrity of the reporting data.

4. Performance and Scalability: Consider the performance and scalability implications of API integration on the data warehouse infrastructure. Ensure that the data warehouse can handle the volume of data retrieved through APIs and scale appropriately to accommodate future growth in data volume and API usage.

5. Real-time Data Integration: Determine whether real-time or near-real-time data integration is required for reporting purposes. WorkForce provides near-real-time streaming APIs. These are not real-time and while they are built on an event-driven architecture, this is not directly available outside of WorkForce.

6. Data Security and Compliance: Implement robust security measures to protect data transmitted via APIs and stored in the data warehouse. This includes authentication, encryption, access controls, and compliance with data privacy regulations such as GDPR or HIPAA, depending on the nature of the data being handled.

7. Monitoring and Logging: Set up monitoring and logging mechanisms to track API usage, data integration processes, performance metrics, errors, and exceptions. This helps identify issues, troubleshoot problems, and optimize API integration and reporting workflows.

8. Data Governance and Metadata Management: Establish data governance policies and metadata management practices to govern the use of API data within the data warehouse. This includes defining data ownership, data lineage, metadata tagging, and version control to ensure data traceability and accountability.

9. Error Handling and Retry Mechanisms: Implement robust error handling and retry mechanisms to handle API failures, network interruptions, and transient errors gracefully. This ensures reliable data retrieval and prevents data loss or inconsistency in reporting.

10. Business Continuity and Disaster Recovery: Develop contingency plans and disaster recovery strategies to mitigate the impact of API outages or disruptions on reporting operations.

By addressing these requirements, organizations can effectively leverage our APIs for reporting in a data warehouse environment, ensuring accurate, timely, and reliable insights from diverse data sources.

Getting Started

To prototype or POC your data integration it is recommended to start with a simpler API and store data in a single table. The swipes API is a good place to start or alternatively the employee

API, ignoring the job sections. Any API which can result in a single data table is recommended as a starting point.

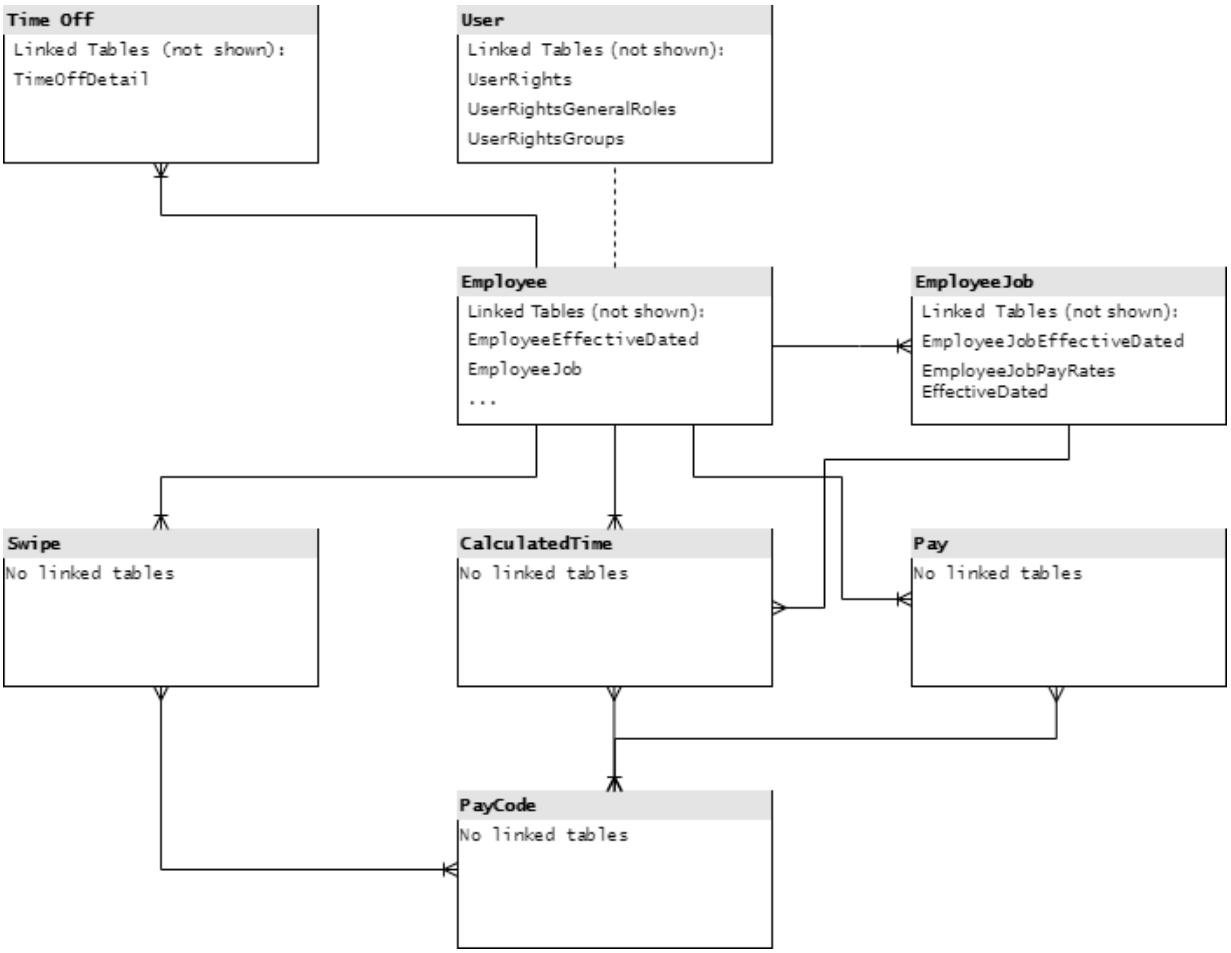
Every Data Feed API includes a field called <object>RecordId (e.g., employeeRecordId). This field is very important as it allows you to update any records that have already come to you over the API with that recordId. It also facilitates deletion where a recordId is tagged as a DELETE.

If you choose to use linked tables, the use of foreign keys (FK) is recommended to help ensure the integrity of the data store and avoid orphaned data.

Data Model

Outline

The data model revolves around employees as shown in this high-level data model.

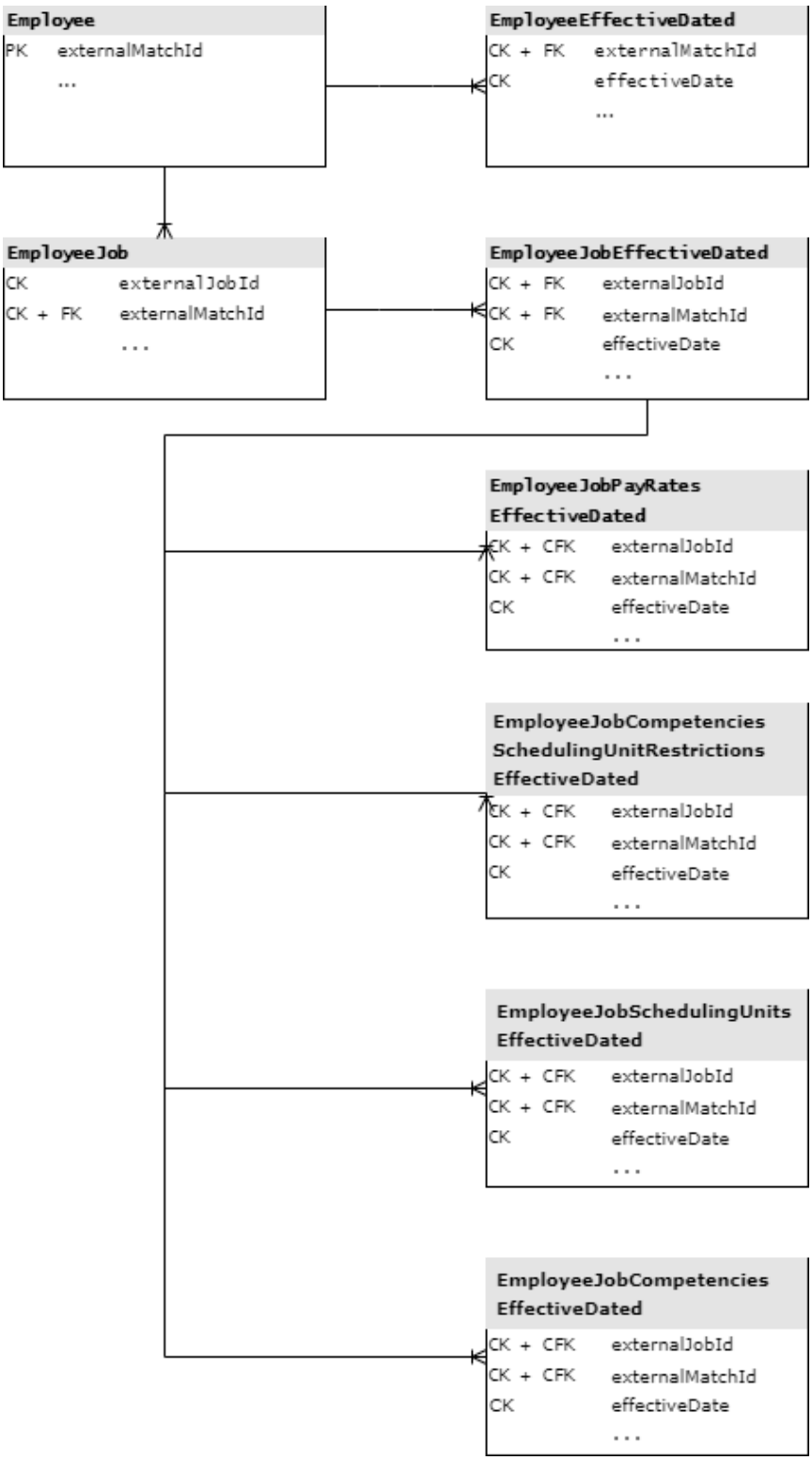


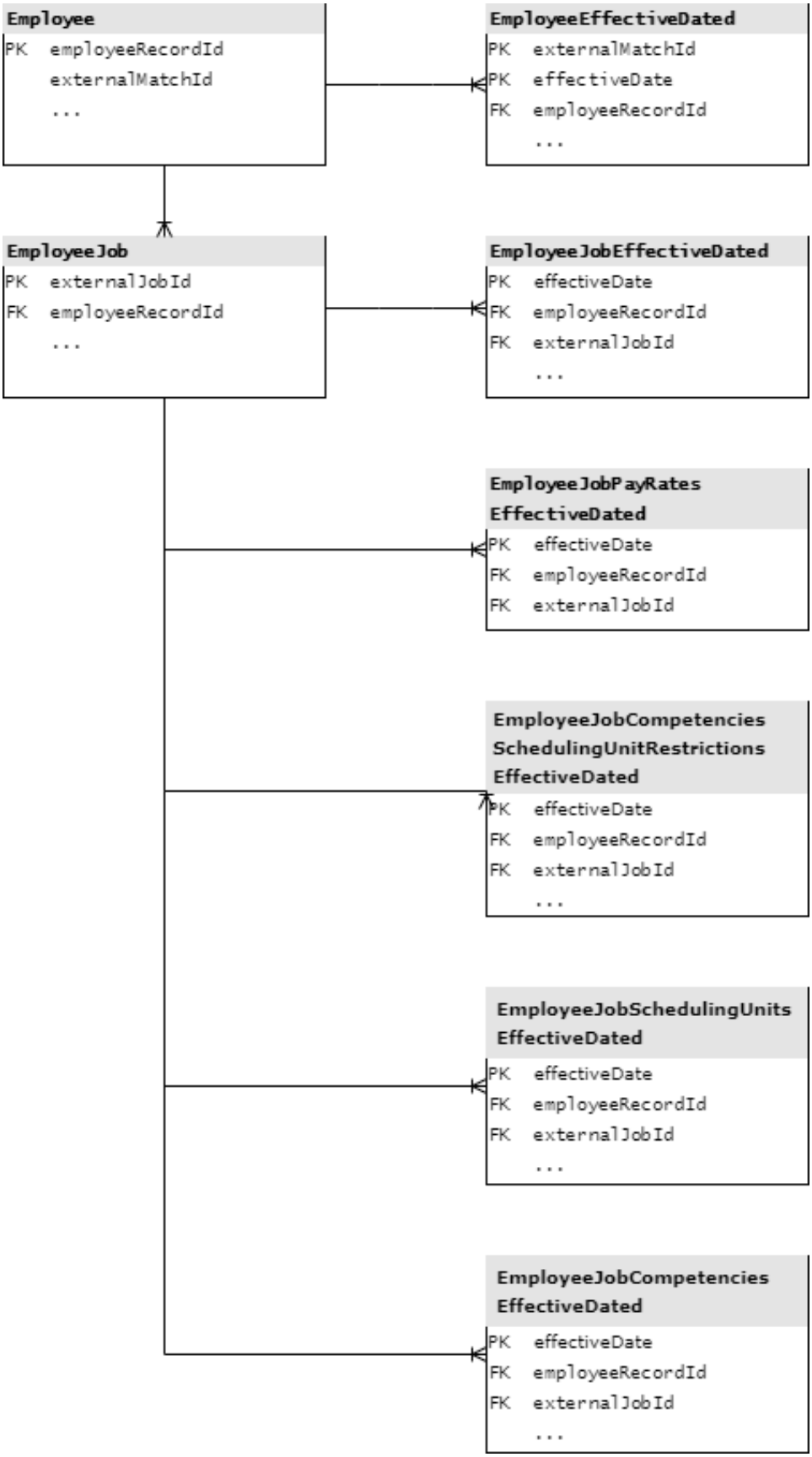
Employee

Employees are people in the system that are scheduled or whose time is tracked; they are not "user only." Only employees can have schedules and timesheets. Anyone who is a user only, cannot.

The Employee API retrieves updates to employee records as they are made in WorkForce. All employee records have a generated `recordId` which can be used as a primary key (PK). In the schema this field maps to `employeeRecordId` for the `recordId` field on the Employee API. The `employeeRecordId` can be used as a Foreign Key in an `EmployeeEffectiveDated` table which contains the full effective dated employee record. A combination of `externalMatchId` and `effectiveDate` can also be used as a composite primary key (PK) for this table, as only one record is held per employee per date of change.

***Note**: externalMatchId is an alternative PK to employeeRecordId in the Employee table if externalMatchId can be guaranteed to be unique. It may be preferred as externalMatchId is customer generated.*


Employee
PK externalMatchId
...
EmployeeEffectiveDated
CK + FK externalMatchId
CK effectiveDate
...
EmployeeJob
CK externalJobId
CK + FK externalMatchId
...
EmployeeJobEffectiveDated
CK + FK externalJobId
CK + FK externalMatchId
CK effectiveDate
...
EmployeeJobPayRates EffectiveDated
CK + CFK externalJobId
CK + CFK externalMatchId
CK effectiveDate
...
EmployeeJobCompetencies SchedulingUnitRestrictions EffectiveDated
CK + CFK externalJobId
CK + CFK externalMatchId
CK effectiveDate
...
EmployeeJobSchedulingUnits EffectiveDated
CK + CFK externalJobId
CK + CFK externalMatchId
CK effectiveDate
...
EmployeeJobCompetencies EffectiveDated
CK + CFK externalJobId
CK + CFK externalMatchId
CK effectiveDate
...


Employee
PK employeeRecordId
externalMatchId
...
EmployeeEffectiveDated
PK externalMatchId
PK effectiveDate
FK employeeRecordId
...
EmployeeJob
PK externalJobId
FK employeeRecordId
...
EmployeeJobEffectiveDated
PK effectiveDate
FK employeeRecordId
FK externalJobId
...
EmployeeJobPayRates EffectiveDated
PK effectiveDate
FK employeeRecordId
FK externalJobId
EmployeeJobCompetencies SchedulingUnitRestrictions EffectiveDated
PK effectiveDate
FK employeeRecordId
FK externalJobId
...
EmployeeJobSchedulingUnits EffectiveDated
PK effectiveDate
FK employeeRecordId
FK externalJobId
...
EmployeeJobCompetencies EffectiveDated
PK effectiveDate
FK employeeRecordId
FK externalJobId
...

Employee

Key	Column Name
PK	externalMatchId
	employeeRecordId
	birthDate
	businessPhoneNumber
	clockPIN
	displayId
	displayName
	emailAddress
	firstName
	lastName
	legalName
	middleNames
	mobilePhoneNumber
	originalHireDate
	phoneNumber
	prefix
	suffix

EmployeeEffectiveDated

Key	Column Name
CK	effectiveDate
CK + FK	externalMatchId
	employeeRecordId
	accrualBand
	accrualDate
	actCaseApproverMatchId
	actCompany
	actEmployeeGroup
	actHireDate
	actLocationCode
	addressLineOne
	addressLineThree
	addressLineTwo

	aggregateFtePct
	aggregatePeriodHours
	aggregateStdDailyHours
	aggregateStdWeeklyHours
	city
	columbiaVacationAccrualDate
	country
	CUSTOM FIELDS
	endEffectiveDate
	gender
	geoFenceLocation
	governmentId
	hrStatus
	importedBadgeId
	isBreak1Waived
	isBreak2Waived
	isCoveredAmericanPetroleum
	isEmployeeDisabled
	isHolidayExempt
	isNightShiftWorker
	isStudent
	isUnder18
	latestHireDate
	leaveCode
	leaveStartDate
	longServiceLeaveReferenceDate
	managedJobsMatchValue
	managerIndicator
	postalCode
	programOfSocialIntegrationId
	seniorityDate
	shiftCode
	sickLeaveCode
	stateOrProvince
	terminationDate

	workerType

EmployeeJob

EmployeeJobs are work assignments. The Employee API retrieves updates to job records as they are made. Job records are also commonly referred to as WT&A assignment records.

EmployeeJob

Key	Column Name
CK	externalJobId
CK + FK	externalMatchId
	employeeRecordId
	jobDescription
	originalStartDate

EmployeeJobEffectiveDated

Key	Column Name
CK	effectiveDate
CK + FK	externalMatchId
CK + FK	externalJobId
	employeeRecordId
	accessLevel
	accrualStartDate
	applicableMinimumWage
	assignmentMonthlyContractHours
	branch
	brazilEmployerId
	businessAddressCity
	businessAddressLineOne
	businessAddressLineThree
	businessAddressLineTwo
	businessAddressPostalCode
	company
	contractType
	costCenter

	country
	CUSTOM FIELDS
	daysPerMonth
	daysPerWeek
	daysPerWeekFuture
	daysPerWeekFutureEffDt
	department
	district
	division
	employerName
	employmentType
	endEffectiveDate
	facilityIdentifier
	flsaStatus
	ftePercent
	holidayCalendarMappingKey
	homeSite
	homeTask
	hourlyBaseRate
	hrStatus
	importedBadgeId
	isActive
	isAlternativeSchedule
	isCasual
	isExcludedFromOtc
	isExemptFromOTCap
	isFullTime
	isPunchingForBreaks
	isQualifiedForSickLeaveAccrual
	isQualifiedForVacationAccrual
	isRehire
	isShiftWorker
	isTaking30MinuteBreaks
	isUsingFlexiTime
	isWeekendsOnly

	isWorkingIrregularSchedule
	jobLevel
	jobRole
	latestStartDate
	locationId
	managersMatchValue
	miniJobberMidiJobber
	nationalRegistryOfLegalEntitiesId
	payCurrency
	payGrade
	payrollId
	payrollSystemId
	payType
	policyProfile
	policyProfileMappingKey
	primary
	programOfSocialIntegrationId
	region
	salariedBaseRate
	scheduleTemplateId
	seniorityDate
	shiftCode
	shiftIndicator
	skillCode
	standardDailyHours
	standardPeriodHours
	standardWeeklyHours
	stateOrProvince
	timeOffApproverMatchId
	timeSheetApproverMatchId
	timezone
	unionCode

EmployeeJobPayRatesEffectiveDated

Key	Column Name
CK + CFK	externalMatchId
CK + CFK	externalJobId
CK	effectiveDate
	employeeRecordId
	frequency
	payGroup
	rate

EmployeeJobCompetenciesSchedulingUnitRestrictionsEffectiveDated

Key	Column Name
CK + CFK	externalMatchId
CK + CFK	externalJobId
CK	effectiveDate
	competencyExternalId
	employeeRecordId
	isDefault
	schedulingUnitExternalId

EmployeeJobSchedulingUnitsEffectiveDated

Key	Column Name
CK + CFK	externalJobId
CK + CFK	externalMatchId
CK	effectiveDate
	schedulingUnitExternalId
	employeeRecordId
	isPrimarySchedulingUnit

EmployeeJobCompetenciesEffectiveDated

Key	Column Name
CK	effectiveDate
CK + CFK	externalMatchId
CK + CFK	externalJobId
	competencyExternalId

	competencyType
	employeeRecordId

CalcTime

CalcTime represents employee time after it has been calculated for employee pay (the Results tab in T&A) but does not imply the pay period has been closed. For example, time worked, breaks, and vacation time are calculated to provide the time records used in the employee's pay. These records are updated every time a timesheet is calculated and are available on the Calculated Time API.

CalcTimeRecords

Key	Column Name
PK	timeRecordId
FK	externalMatchId
	comments
	effectiveRate
	employeeJob
	endTimestamp
	externalJobId
	grossPay
	hours
	payCode
	recordType
	startTimestamp
	workDate

Pay

Pay represents all of the calculated time that exists for the job on the date and is deliberately unfiltered, as there is no way of knowing which time slices are actually important and which are not.

This is the data that is intended to drive payroll, not necessarily the final pay data itself. (If WT&A is doing all of the final pay calculations, then it very well could be, but it could also just be the raw calculated data to be fed into another system to actually compute the final pay information.)

Ultimately the only real difference between the Calculated Time and Calculated Pay datasets is when the data is published and what sort of data is included. Calculated Time is published every time a period is saved or calculated and includes the information that falls within that period only. Calculated Pay is only published when timesheets are locked (and gets retracted when the timesheets are unlocked), so it is only present for a period when that period is in a state "pending payroll processing", essentially. Furthermore, the Calculated Pay includes not just the calculated time for that period, but also any adjustments stemming from amendments to prior periods that should be paid out with this period. (So it is fully possible, for instance, to have a periodStartDate of 2022-08-22. a periodEndDate of 2022-08-28, and a workDate of 2022-04-13.). This data is available on the Pay API.

Pay

Key	Column Name
PK	PayRecordId
FK	externalMatchId
	ADDITIONAL FIELDS
	amount
	comments
	effectiveRate
	employeeJob
	endTimestamp
	externalJobId
	grossPay
	hours
	payCode
	payCurrencyCode
	periodEndDate
	periodStartDate
	recordType
	startTimestamp
	workDate

PayCode

Pay codes are retrieved from the Policy Set API. They represent the pay codes that have been defined and configured in WorkForce. Note that the Policy Set API also provides banks. An appropriate filter should be applied to ensure only pay codes are stored in PayCode. Also note that there is currently no requirement for all pay codes to be specified in at least one pay code

policy set in WT&A configuration. As such, pay codes should also be retrieved from calculated time and pay as there can be pay codes in these records that are not in a policy set.

PayCode

Key	Column Name
PK	payCodeId
	payCodeLabel
	payCodePolicySetName
	payCodePolicySetPolicyType
	payCodePolicySetRecordId

Swipe

Swipes are a record of processed clock events that have been collected from WorkForce Software compatible devices. Manual adjustments are not included.

Swipe

Key	Column Name
PK	swipeRecordId
	clockId
	clockType
	externalMatchId
	jobId
	locationLatitude
	locationLongitude
	paycode
	swipeProcessedTime
	swipeTime
	type

TimeOff

Contains information regarding an employee's time off requests.

TimeOff

Key	Column Name
PK	timeOffRecordId
	comments
	endDate
	externalMatchId
	requestMadeAt
	startDate
	status
	timeOffType

TimeOffDetail

Key	Column Name
FK	timeOffRecordId
	date
	endDateTime
	payCode
	quantity
	startDateTime
	unit

User

Users in WorkForce Software are anyone who can login to the system. They may also be employees but do not have to be. User data is retrieved from the User API.

User

Key	Column Name
PK	userId
	displayName
	enabled
	externalMatchId
	firstName
	lastName
	legalName

	loginId
	middleNames
	prefix
	ssoMatchId
	suffix
	userRecordId

UserRights

UserRights represent the actions of users in the system. Rights can be derived directly from their role e.g., TA-WEBCLOCK_USER or from a group to which they belong which itself can also have a role.

General Roles are roles that define what actions a user can take with respect to their data. Group Roles are roles that define what actions a user can take with respect to the jobs of employees in an employee group they have access to.

UserRights

Key	Column Name
PK	userRightsRecordId
FK	userId
	externalMatchId

UserRightsGeneral Roles

Key	Column Name
FK	userRightsRecordId
	generalRole

UserRightsGroups

Key	Column Name
FK	userRightsRecordId
	groupId
	roleId
	type

Using the Data Enrichment API

The Data Enrichment API allows for custom mappings and transformations to be specified for a customer account that will modify the data being received by the Person API. This allows for derivations of additional values that aren't present in the original source data and for transforming values in the source data that represent the data that we need in a different fashion than we actually need. The Data Enrichment API also allows for specifying account-specific validations, which allows person data to be rejected if it doesn't meet the requirements for that account even if the data would otherwise meet all of the general requirements for importing person data.

Authentication and Authorization

Authentication and Authorization uses the same process and protocols as for other WorkForce REST APIs i.e., OAuth 2.0 over HTTPS. Each call to the API requires an Access Token. Tokens are retrieved from the Auth API using your Secret Key. See the Authentication and Authorization section of this guide for more details.

Viewing a List of Existing Enrichment Definitions

Existing Data Enrichment definitions for an account can be listed and filtered by name. To view a list of existing definitions, a request should be made to the `/data-enrichment/list/v1` endpoint. The results can be filtered by `name` and then further filtered by `schemaVersion` if desired. There is no support to only filter by `schemaVersion` without also providing a `name` as this provides no value.

Sample response for call to the `/data-enrichment-list/v1` endpoint without filter parameters returning a list of the current active schemas:

```
GET /data-enrichment-list/v1

[
   {
      "schemaVersion": 4,
      "name": “test”,
      “effectiveDate”: “1900-01-01”,
      “endEffectiveDate”: “3000-12-31”
   },
   {
      "schemaVersion": 2,
      "name": “anotherTest”,
      “effectiveDate”: “1900-01-01”,
      “endEffectiveDate”: “2023-12-31”
   },
   {
      "schemaVersion": 2,
      "name": “anotherTest”,
```

```
        “effectiveDate”: “2024-01-01”,
        “endEffectiveDate”: “3000-12-31”
    }


]
```

Sample response for call to the `/data-enrichment-list/v1` endpoint filtered by `name` returns the current active schema with that name, including effective dated schemas if there are any:

```
GET /data-enrichment-list/v1?name=anotherTest
```

```
[
    {
        "schemaVersion": 2,
        "name": “anotherTest”,
        “effectiveDate”: “1900-01-01”,
        “endEffectiveDate”: “2023-12-31”
    },
    {
        "schemaVersion": 2,
        "name": “anotherTest”,
        “effectiveDate”: “2024-01-01”,
        “endEffectiveDate”: “3000-12-31”
    }
]
```

It may be useful to see a previous version. Sample response for call to the `/data-enrichment-list/v1` endpoint filtered by `name` and `schemaVersion`:

```
GET /data-enrichment-list/v1?name=anotherTest&schemaVersion=1
```

```
[
    {
        "schemaVersion": 1,
        "name": “anotherTest”,
        “effectiveDate”: “1900-01-01”,
        “endEffectiveDate”: “3000-12-31”
    }
]
```

Viewing an Existing Enrichment Definition

Existing Data Enrichment definitions for an account can be viewed by `name`, `schemaVersion` and `effectiveDate`. All three parameters must be provided. To view the detail of an existing definition, a request should be made to the `/data-enrichment/v1` endpoint providing the `name, schemaVersion and effectiveDate` as parameters. The results will show the detail for the enrichment matching that combination of `name`, `schemaVersion` and `effectiveDate`.

Sample response for call to the /data-enrichment/v1 endpoint providing name, schemaVersion and effectiveDate:

```
/data-enrichment/v1?name=testName&schemaVersion=1&effectiveDate=1900-01-01
```

```
{
  "name": "testName",
  "schemaVersion": 1,
  "effectiveDate": "1900-01-01",
  "endEffectiveDate”: “3000-12-31"
  "rank": "1",
  "collection": "/assignments",
  "value": "40",
  "evaluate": true,
  "roundingPlace": "2",
  "condition": "'${standard_daily_hours}'.isEmpty()",
  "isDefault": true,
  "isValidation": false,
  "validationMessage": "not a validation",
  "destination": "testDestination",
  "destinationType": "number",
  "destinationDateFormat": "MM/dd/YYYY",
  "sources": {
    "displayName": {
      "path": "/name/displayName",
      "stringFormatter": "",
      "dateFormat": ""
    }
  }
}
```

`schemaVersion` and `endEffectiveDate` should both always be included in the GET response but cannot be specified in the PUT request to create an enrichment.

The response returned can be used as the starting point for the creation of an updated enrichment called “testName” by making a PUT request to `the /data-enrichment/v1` endpoint. The testName enrichment will automatically have its `schemaVersion` incremented.

Creating a New Enrichment Definition

To create a new Data Enrichment definition, the `/data-enrichment/v1` endpoint should be used. Requests sent to this endpoint are expected to be of the following form:

Sample request format

```
{
    "sources": {
        "asource": {
            "path": "path1",
            "formatter": "formatter1",
            "sourceDateFormat": "yyyy-MM-dd"
        },
        "anothersource": {
            "path": "path2",
            "formatter": "formatter2",
```

```
            "sourceDateFormat": "yyyy-MM-dd"
        }
    },
    "effectiveDate": "2012-10-20",
    "rank": 1,
    "name": "testName",
    "collection": "anothercollection",
    "value": "someValue",
    "evaluate": true,
    "roundingPlace": 2,
    "condition": "!!boolean",
    "validationMessage": "someMessage",
    "destination": "aDestination",
    "destinationType": "STRING",
    "destinationDateFormat": "yyyy-MM-dd"
}
```

If a valid request is made to the `/data-enrichment/v1` endpoint with a `name` that does not already exist for the account, then a new Data Enrichment definition will be created with that name that will apply for that account.

Data Enrichment definitions are effective-dated and will only apply to person data that falls within the effective date range of the definition. Ending effective dates are not supplied as part of the request; instead, the ending date is assumed to be the end of time for all incoming requests. So, a definition with an effective date of 2023-09-06 would apply to all person data that is effective from 2023-09-06 onward.

The following properties are required on all requests:

- `name`: the unique name for the Data Enrichment definition. This acts as a key for identifying the definition for later updates. Names can contain spaces for human readability if desired.
- `effectiveDate`: the starting date on which the definition takes effect. All new definitions are assumed to run through the end of time, though this can be changed via later updates to the definition (see below)
- `rank`: this defines the order in which different Data Enrichment definitions effective on a given date will be applied. The specific values here have no intrinsic meaning, but definitions with a lower rank will be applied before definitions with a higher rank. This allows for control over the ordering of the application of the different definitions, which is needed if later definitions are relying on the result of transformations applied by earlier definitions in order to function correctly.
- `evaluate`: true if the value should be evaluated as an expression, false if it should not. Values are evaluated using the Spring Expression Language (SpEL).

- `isDefault`: indicates if this definition is intended to provide a default value for a property in the person data if that property is not populated in the person request.
- `isValidation`: indicates if this definition is intended for validation purposes (as opposed to transformation purposes). A value of `true` here means that the definition will be compared against the person data, and if it does not meet the defined condition then the person data will be rejected. A value of `false` means that the definition will be used to modify the person data.
- `sources`: this is mapping from a name for a source to information about that source for all of the different source fields that should be evaluated by the definition. At least one source must be defined for each Data Enrichment definition. Each entry in this mapping can have the following properties:
    - `path`: the location of the source field in the incoming request data
    - `stringFormatter`: a string formatter that should be applied to the values found in the source. (This is optional and should only be specified if formatting of string values is desired.) Formatter is expected to conform to the formatting rules defined here.
    - `dateFormat`: a date format that should be applied to values found in the source. (This is optional and should only be specified if the source represents date values.) Date formats are expected to conform to the formatting rules defined here.

The following additional properties are optional, and can further impact the behavior of the Data Enrichment definition if desired:

- `collection`: specifies an array of data in the person data that should be evaluated against this definition
- `value`: the value that should be applied by the transformation
- `roundingPlace`: defines the precision that should be used for rounding floating-point values
- `condition`: the condition that should be evaluated to determine if the transformation should be applied or if the data passes the custom validation
- `validationMessage`: the error message that should be reported if the person data being evaluated fails the validation defined by the definition
- `destination`: specifies the field that should be updated as a result of the transformation
- `destinationType`: specifies what type of values the destination field is expected to contain. Valid values are `string`, `number`, `boolean`, `date`, and `timestamp`

- `destinationDateFormat`: specifies what date format should be used when writing to the destination field, if the destination field is storing date values

Updating an Existing Enrichment Definition

To update an existing Data Enrichment definition, simply send a new request in using the same `name` value that is present on the definition you wish to update. Updates can take one of two forms:

Updating an Existing Enrichment Definition Record

To update an existing Data Enrichment definition record, so that a new behavior will apply for the entire effective date range of that record, simply send through a new request that has both the same `name` and `effectiveDate` that exist on the record you wish to update. This will cause the existing record to be updated for its entire effective date range, completely replacing the original definition with the definition in the new request. (Note that any effective-dated changes to that definition that were effective after the date of the record you are updating will be removed, so if you want to update a historical record in the effective-dated stack and also preserve later effective-dated variants of that definition, those later effective-dated records will need to be sent through again as well—in chronological order from oldest to most recent).

Creating an Effective-Dated Change to an Enrichment Definition

To update an existing Data Enrichment definition so that its behavior changes on a given effective date, simply send through a new request that has the same `name` value but with a different `effectiveDate` than was present on the original definition. This will cause the existing definition to be end effective dated on the day before the new effective date, with the new definition taking effect starting on the specified effective date and running through the end of time. (Note that if the new `effectiveDate` provided is before the original definition's effective date, then the original definition will be removed as part of this process.)

Common Use Cases

The following sections provide examples of how a Data Enrichment definition can be defined to accomplish several different common tasks:

Setting a Field to a Calculated Value

The following request will create a Data Enrichment definition that populates the `displayName` attribute of the incoming person data by concatenating the provided `firstName` and `lastName` values together (with a space in between):

Setting a field to a calculated value

```
{
  "name": "deriveDisplayName",
  "effectiveDate": "2023-09-06",
  "rank": "1",
  "value": "${FIRST} ${LAST}",
  "evaluate": false,
  "isDefault": false,
  "isValidation": false,
  "destination": "/name/displayName",
  "destinationType": "STRING",
  "sources": {
    "FIRST": {
      "path": "/name/firstName"
    },
    "LAST": {
      "path": "/name/lastName"
    }
  }
}
```

These derivations can also be applied conditionally. For instance, the following request will create a Data Enrichment definition that populates the `displayName` attribute of the incoming person data by concatenating the provided `firstName` and `lastName` values together (with a space in between), but only if the `displayName` does not already contain a value:

Conditionally setting a field to a calculated value

```
{
  "name": "deriveDisplayName",
  "effectiveDate": "2023-09-06",
  "rank": "1",
  "value": "${FIRST} ${LAST}",
  "evaluate": false,
  "isDefault": false,
  "isValidation": false,
  "destination": "/name/displayName",
  "destinationType": "STRING",
  "condition": "'${DISPLAY_NAME}'.isEmpty()",
  "sources": {
    "FIRST": {
      "path": "/name/firstName"
    },
    "LAST": {
      "path": "/name/lastName"
    },
    "DISPLAY_NAME": {
      "path": "/name/displayName"
    }
  }
}
```

Checking If a Field is Empty

The following request will create a Data Enrichment definition that checks if the `timeZone` property on the job record is empty and will error out the incoming person data if it is.

Checking if a field is empty

```
{
  "name": "timeZoneIsEmpty",
  "effectiveDate": "2023-09-06",
  "rank": "1",
  "evaluate": false,
  "isDefault": false,
  "isValidation": true,
  "validationMessage": "Time zone was empty",
  "condition": "'${TIME_ZONE}'.isEmpty()",
  "sources": {
    "TIME_ZONE": {
      "path":
"/employee/jobs/*/effectiveDatedJobInfo/${effectiveDate=@EFF_DT}/timeZone"
    }
  }
}
```

Checking If a Field Contains a Specified Value

The following request will create a Data Enrichment definition that checks if the `payType` property on the job record contains a value of either “BAD_VALUE1”, “BAD_VALUE2”, or “BAD_VALUE3” and will error out the incoming person data if it is.

Checking if a field contains a specified value

```
{
  "name": "invalidPayType",
  "effectiveDate": "2023-09-06",
  "rank": "1",
  "evaluate": false,
  "isDefault": false,
  "isValidation": true,
  "validationMessage": "Pay type contained invalid value",
  "condition": "{'BAD_VALUE1', 'BAD_VALUE2', 'BAD_VALUE3'}.contains('${PAY_TYPE}')",
  "sources": {
    "PAY_TYPE": {
      "path":
"/employee/jobs/*/effectiveDatedJobInfo/${effectiveDate=@EFF_DT}/payType"
    }
  }
}
```

Rounding a Numeric Value

The following request will create a Data Enrichment definition that will round the value provided in the job's `standardDailyHours` property to two decimal places before it is stored in the person data.

Rounding a numeric value

```
{
  "name": "roundStandardDailyHours",
  "effectiveDate": "2023-09-06",
  "rank": "1",
  "value": "${HOURS}",
  "roundingPlace": "2",
  "evaluate": true,
  "isDefault": false,
  "isValidation": false,
  "destination":
"/employee/jobs/${externalJobId=@JOB_ID}/effectiveDatedJobInfo/${effectiveDate=@EFF_D
T}/standardDailyHours",
  "destinationType": "NUMBER",
  "sources": {
    "HOURS": {
      "path":
"/employee/jobs/${externalJobId=@JOB_ID}/effectiveDatedJobInfo/${effectiveDate=@EFF_D
T}/standardDailyHours"
    }
  }
}
```

Working with Dates

The following request will create a Data Enrichment definition that maps the `EffectiveStartDate` property in the source data to the `originalStartDate` on all of the different job records.

Working with dates

```
{
  "name": "populateOriginalStartDate",
  "effectiveDate": "2023-09-06",
  "rank": "1",
  "evaluate": false,
  "isDefault": false,
  "isValidation": false,
  "destination": "/employee/jobs/*/originalStartDate",
  "destinationType": "DATE",
  "sources": {
    "START_DATE": {
      "path": "/EffectiveStartDate",
      "stringFormatter": "%tF",
```

```
      "dateFormat": "dd/MM/yyyy"
    }
  }
}
```

Updating Data on a Specific Job Only

The following request will create a Data Enrichment definition that maps the `primary` value for all effective-dated records belonging to the same job.

Updating data on a specific job only

```
{
  "name": "populatePrimary",
  "effectiveDate": "2023-09-06",
  "rank": "1",
  "collection": "/assignments",
  "value": "${PRIMARY}",
  "evaluate": false,
  "isDefault": false,
  "isValidation": false,
  "destination":
"/employee/jobs/${externalJobId=${JOB_ID}}/effectiveDatedJobInfo/*/primary",
  "destinationType": "BOOLEAN",
  "sources": {
    "PRIMARY": {
      "path": "/assignments/@INDEX/PrimaryAssignmentFlag"
    },
    "JOB_ID": {
      "path": "/assignments/@INDEX/AssignmentNumber"
    }
  }
}
```

In this example, we are defining two sources, `JOB_ID` and `PRIMARY`, in the source data which are capturing the external match ID of the job currently being evaluated and whether that job is marked as primary or not. The `destination` is then defined to match all of the effective-dated records for the job where the `externalJobId` matches that external match ID for the job, with the `primary` property on those records being set to the boolean value being read from the source data.

Testing an Enrichment Definition

An existing testing endpoint has been set up to see how a data enrichment would be applied to a request without actually sending the data to Kafka.

The endpoint is at `/data-enrichment/v1/test`. To use this endpoint, you would send a GET request to the endpoint, but also include a PERSON API PUT request in the body. The API will return the request you just made with all the enrichment transformations applied to it.

For example, if you wanted to test the example given in the **Setting a Field to a Calculated Value** section above which concatenates the first and last names to the display name, you would put the following request in the body.

```
{
  "externalMatchId": "someId",
  "name": {
      "firstName": "first",
      "lastName": "last",
      "displayName": ""
  }
}
```

After sending it to the endpoint, the API would return something like:

```
 {
  "externalMatchId": "someId",
  "name": {
      "firstName": "first",
      "lastName": "last",
      "displayName": "first last"
   }
 }
```

Appendix A: Field Mappings for Person v2

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
externalMatchId	EXTERNAL_HR_ID COMPUTED_MATCH_ID		StaffId / StaffWorkId	id that uniquely identifies the person in the customer's system. Typically, their employee ID	String		256		*
name **> firstName**	FIRST_NAME		FirstName / GivenName / LegalGivenName	The first name.	String	alphabetic or -'() characters	80		
name **> middleNames**	MIDDLE_NAME		MiddleName / OtherNames / LegalOtherNames	The middle names.	String	alphabetic or -'() characters	80		
name **> lastName**	LAST_NAME		LastName / FamilyName / LegalFamilyName	The last name.	String	alphabetic or -'() characters	80		
name **> prefix**			Prefix		String		20		
name **> suffix**			Suffix		String		20		
name **> legalName**				formatted legal name, as opposed to a shortened version/nickname	String		1024		
name **> displayName**	DISPLAY_NAME		DisplayName	(unique) name displayed to users so that person can be identified	String		255		Required if name is supplied
emailAddress	EMAIL_ADDRESS		Email	Email address of person.	Email		128		
birthDate	BIRTH_DATE	{MEX_EMP_IMPORT _MAPPINGS}	DOB	The date of birth of person.	Date				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee				The object representing the entire set of employee and job data for the person.					Required of user is not supplied
employee > displayId	DISPLAY_EMPLOYEE			Unique ID for this employee that an end-user would recognize	String		256		*
employee > originalHireDate	ORIGINAL_HIRE_DATE			The ISO-8601-formatted date that the employee originally began employment.	Date				
employee > phoneNumber	PHONE1			E.164-formatted phone number for the employee	Phone		15		
employee > businessPhoneNumber	BUSINESS_PHONE			E.164-formatted business phone number for the employee	Phone		15		
employee > mobilePhoneNumber	MOBILE_PHONE			E.164-formatted mobile phone number for the employee	Phone		15		
employee > clockPin	CLOCK_PIN			PIN for alternative punch method for people who are unable to use the biometric readers	String		40		
employee > isFullEmployeeHistory				True if the effectiveDatedEmployeeInfo array being provided reflects the entire effective-dated history that should exist for the employee, false if the effectiveDatedEmployeeInfo being provided reflects only a partial effective-dated history for the employee	Boolean		15		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee > effectiveDatedEmployeeInfo				The employee record associated with a specific effective date for which it is active.					*
employee > effectiveDatedEmployeeInfo > effectiveDate	EFF_DT			The effective date of the employee's data in the record.	Date			System date on load	*
employee > effectiveDatedEmployeeInfo > hrStatus	HR_STATUS			Status code of the employee in the HR system	String		20		
employee > effectiveDatedEmployeeInfo > latestHireDate	LATEST_HIRE_DATE			The effective dated latest hire date for any job. This date does not drive actual active/inactive status, but is captured just for use in calculations and for reporting purposes	Date				
employee > effectiveDatedEmployeeInfo > accrualDate	ACCRUAL_HIRE_DATE			The effective dated accrual date for all jobs. The date that accrual calculations should be based on. Used to provide an adjusted date for calculating accrual service ranges that is different than the actual hire date.	Date				
employee > effectiveDatedEmployeeInfo > terminationDate	TERMINATION_DATE			The most recent termination date for the employee, as of the effective date, if the employee is not active. Represents the employee's last active date preceding the termination. This date does not drive actual active/inactive status at all, but is captured just	Date				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
				for use in calculations and for reporting purposes					
employee > effectiveDatedEmployeeInfo > seniorityDate	SENIORITY_DATE			The ISO-8601-formatted dated that describes the calculation of a person's seniority. This is used for PTO accruals	Date				
employee > effectiveDatedEmployeeInfo > leaveStartDate	LEAVE_START_DATE			The ISO-8601-formatted date that the leave began for the employee	Date				
employee > effectiveDatedEmployeeInfo > leaveCode	STATUS_CODE2			Designates the type of leave for the employee. Used to drive leave calculations in certain configurations	String		2000		
employee > effectiveDatedEmployeeInfo > governmentId	GOVERNMENT_ID			Government identifier for the employee	String		20		
employee > effectiveDatedEmployeeInfo > gender	GENDER		Gender	Effective dated gender	String		64		
employee > effectiveDatedEmployeeInfo > managedJobsMatchValue	MANAGER_MATCH_FIELD			Value indicating which jobs this employee manages, which matches the value of the 'managersMatchValue' on the jobs that are managed by this employee. This value could be the manager's ID, a location ID, etc. While this value intrinsically makes sense to be stored as part of the user data, rather than the	String		2000		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
				employee, it needs to be mapped at the employee level in order to function correctly with legacy WT&A behavior (in particular, standard reports). This is temporary until a more permanent rights and permissions framework is established.					
employee > effectiveDatedEmployeeInfo > homeAddress > country	HOME_ADDRESS_COUNTRY			The 3 Character Country Code for this employee.	Country Code		60		
employee > effectiveDatedEmployeeInfo > homeAddress > stateOrProvince	HOME_ADDRESS_STATE_PROVINCE			The employee's home state/province in ISO 3166-2 format.	State/Province		40		
employee > effectiveDatedEmployeeInfo > homeAddress > postalCode	HOME_ADDRESS_POSTAL_CODE			The Postal Code of the home address	String		40		
employee > effectiveDatedEmployeeInfo > importedBadgeId	IMPORTED_BADGE_ID			Imported badge ID used to generate a badge record when badge imports are run	String		2000		
employee > effectiveDatedEmployeeInfo > managerIndicator	MANAGER_INDICATOR			Flag defining whether the employee is a manager	Boolean				
employee > effectiveDatedEmployeeInfo > shiftCode	SHIFT_CODE			Home shift code for the employee. Used to identify a shift for use in pay calculations	String		85		
employee > effectiveDatedEmployeeInfo > homeAddress > city	HOME_ADDRESS_CITY			The city that the home address resides in	String		128		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee > effectiveDatedEmployeeInfo > homeAddress > city	HOME_ADDRESS_LINE_1			Home Address Line 1	String		60		
employee > effectiveDatedEmployeeInfo > homeAddress > lineOne	HOME_ADDRESS_LINE_1			Home Address Line 1	String		60		
employee > effectiveDatedEmployeeInfo > homeAddress > lineTwo	HOME_ADDRESS_LINE_2			Home Address Line 2	String		60		
employee > effectiveDatedEmployeeInfo > homeAddress > lineThree	HOME_ADDRESS_LINE3			Home Address Line 3	String		60		
employee > effectiveDatedEmployeeInfo > templateEmployeeFields	These properties are passed through specifically to WT&A for use with its Templates.								
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > accrualBand	ACCRUAL_BAND	{EXT_EU_ANNUAL_LEAVE}{SAU_VAC_BANK}		Required if accruals are based on the employee's age range or accrual band.	Integer				
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > actCompany	ACT_COMPANY	{ABSENCE_TRACKER}{EXT_ACT }		ACT Company.	String		85		
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > actEmployeeGroup	ACT_EMPLOYEE_GROUP	{ABSENCE_TRACKER}{EXT_ACT }		ACT Employee Group.	String		85		
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > actHireDate	ACT_HIRE_DATE	{ABSENCE_TRACKER}{EXT_ACT }		The date that describes the employee's ACT hire date.	Date				
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > actLocationCode	ACT_LOCATION_CODE	{ABSENCE_TRACKER}{EXT_ACT }		This is the employee's state of employment, but can include city or county where applicable leave	String		85		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
				programs are supported by ACT.					
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > actCaseApproverMatchId	ACT_CASE_APPROVER_MATCH_ID	{ABSENCE_TRACKER}{EXT_ACT }		Match ID of the approver of the employee's ACT cases	String		2000		
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > aggregateFtePct	AGGREGATE_FTE_PCT	{ABSENCE_TRACKER}{EXT_ACT }		Full-time equivalent percentage.	Number	0-100			
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > aggregateStdDailyHours	AGGREGATE_STD_DAILY_HOURS	{ABSENCE_TRACKER}{EXT_ACT }		Aggregate Standard Daily Hours.	Number				
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > aggregateStdWeeklyHours	AGGREGATE_STD_WEEKLY_HOURS	{ABSENCE_TRACKER}{EXT_ACT }		Aggregate Standard Weekly Hours	Number				
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > aggregatePeriodHours	AGGREGATE_STD_PERIOD_HOURS	{ABSENCE_TRACKER}{EXT_ACT }		Aggregate Standard Period Hours	Number				
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > columbiaVacationAccrualDate	EMPLOYEE_ACCRUAL_DATE	{COL_STAT_VAC_BANK}		The date that describes when the anniversary of the monthly accrual will occur.	Date				
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > geoFenceLocation	GEO_FENCE_LOCATION	{EMP_IN_OUT_PUNCH_OUTSIDE_GEOFENCE}{EMP_IN_OUT_PUNCH_OUTSIDE_RADIUS}		Contains a field match value required for the geo-fence zone data lookup.	String		2000		
employee > effectiveDatedEmployeeInfo > templateEmployeeFields > isBreak1Waived	MEAL_BREAK_WAIVER_1	{US_WAGE_HOUR_AND_HOL}		Flag to indicate whether this employee has waived a meal break.	Boolean				
employee > effectiveDatedEmployeeInfo	MEAL_BREAK_WAIVER_2	{US_WAGE_HOUR_AND_HOL}		Flag to indicate whether this employee	Boolean				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
> templateEmployeeFields **> isBreak2Waived**				has waived a meal break.					
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> isCoveredAmericanPetroleum**	COVERED_NONCOVERED_STATUS	{FM_PIP_SA}{FM_PIP_SA_DATA_ELEMENTS}		The employee's covered status - with respect to American Petroleum Institute's RP-755 guidelines.	Boolean				
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> isEmployeeDisabled**	DISABILITY_FLAG	{POL_ANNUAL_LEAVE} {DEU_STAT_VAC_BANK}{DEU_EMP_IMPORT_MAPPINGS}		Flag that indicates employees who are eligible to the Disability Vacation / have a severe disability.	Boolean				
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> isHolidayExempt**	SUSPEND_ACCRUAL_FLAG	{FIN_ANNUAL_HOL}{DEU_STAT_VAC_BANK}{DNK_ANNUAL_HOLIDAY}{EXT_EU_ANNUAL_LEAVE}{NZL_ANNUAL_HOLIDAY}{SAU_VAC_BANK}		Flag to indicate whether an employee is temporarily exempt from Annual Holiday Accrual.	Boolean				
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> isNightShiftWorker**	NIGHT_SHIFT_WORKER_FLAG	{GERMANY_NIGHT_SHIFT_PAY}{EU_NIGHT_WORKER}		Flag that indicates Night Worker employees who are subject to specific rules.	Boolean				
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> isStudent**	STUDENT_FLAG	{CGT_EU_HOURS_OF_WORK}{DEU_EMP_IMPORT_MAPPINGS}		Flag to indicate Student status.	Boolean				
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> isUnder18**	UNDER_18_FLAG	{CGT_HOURS_WORKED_OVER_THRESHOLD}{DEU_EMP_IMPORT_MAPPINGS}{CGT_EU_DAILY_REST_BREAK}{CGT_EU_WEEKLY_REST_BREAK}{CGT_EU_REST_BREAK}{THA_HOURS_OF_WORK}{THA_DAILY_REST_BREAK}{THA_WEEKLY_HOLIDAY}		Flag to indicate Under 18 status.	Boolean				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> longServiceLeaveReferenceDate**	ENTITLEMENT_REFERENCE_DATE	{AUS_LSL}		The date that describes the starting point for determining the date on which an employee may use Long Service Leave they have accrued.	Date				
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> programOfSocialIntegrationId**	PIS_CNPJ	{BRA_EMP_IMPORT_MAPPINGS}		The employee's Program of Social Integration (PIS) identifier	String		2000		
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> sickLeaveCode**	SICK_LEAVE_CODE	{CGT_US_SICK_LEAVE_BASE}{CGT_US_SICK_LEAVE}		When multiple sick leave programs are implemented, used to designate which sick leave program to apply to the employee when defining the 'Rule Eligibility' criteria.	String		2000		
employee **> effectiveDatedEmployeeInfo** **> templateEmployeeFields** **> workerType**	WORKER_TYPE	{MEX_EMP_IMPORT_MAPPINGS}		Defines whether an employee is a Basic Worker, Temporary Worker or a Trusted Worker.	String		2000		
employee **> effectiveDatedEmployeeInfo** **> customFields** **> name => value**			Custom	Custom fields are listed as key value pairs with a name and a value.	String => String		varies		name is required for any custom field
employee **> jobs** **> externalJobId**	COMPUTED_MATCH_ID			ID that uniquely identifies this job for the employee. externalJobIds within an employee must be unique	String		256		*
Employee **> jobs** **> isFullJobHistory**				True if the effectiveDatedJobInfo array being provided reflects the entire effective-dated history	Boolean				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
				that should exist for the job, false if the effectiveDatedJobInfo being provided reflects only a partial effective-dated history for the job					
employee **> jobs** **> jobDescription**				The text displayed to an end user to know which job is being referenced	String		256		*
employee **> jobs** **> originalStartDate**	ORIGINAL_ASSIGNMENT_BEGIN_DATE		StartDate	The date that the employee originally began this job. This date does not drive actual active/inactive status at all, but is captured just for use in calculations and for reporting purposes	Date				
employee **> jobs** **> effectiveDatedJobInfo**	The job record associated with a specific effective date for which it is active.								
employee **> jobs** **> capabilities** **> externalId** (Note: Deprecated. Use Competencies instead.)				ID that uniquely identifies the capability in the customer's systems	String				*
employee **> jobs** **> capabilities** **> description** (Note: Deprecated. Use Competencies instead.)				The text displayed to an end user to know which capability is being referenced	String				*
employee **> jobs** **> capabilities** **> primary** (Note: Deprecated. Use Competencies instead.)				boolean to denote default/primary capability	Boolean				*

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee **> jobs** **> effectiveDatedJobInfo** **> effectiveDate**	EFF_DT			The date that the data in this object is effective on	Date				
employee **> jobs** **> effectiveDatedJobInfo** **> endDate**	ASSIGNMENT_END_DATE		LeaveDate ?	The date that the employee stopped working on this job.	Date				
employee **> jobs** **> effectiveDatedJobInfo** **> primary**	PRIMARY_ASGNMT			Boolean to denote default/primary job	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> isActive** Negates the value sent in (if they aren't active, they are soft terminated	SOFT_TERMINATED			The status of the job as of the effective date. True if the job is active for the effective-dated range, false otherwise	Boolean				*
employee **> jobs** **> effectiveDatedJobInfo** **> hrStatus**	HR_STATUS			Status code of the job in the HR system	String		20		
employee **> jobs** **> effectiveDatedJobInfo** **> scheduleTemplateId**	SCHEDULE_TEMPLATE			Unique identifier for the schedule template being assigned. Must match a schedule template policy ID in time and attendance	String		85		
employee **> jobs** **> effectiveDatedJobInfo** **> shiftCode**	SHIFT_CODE			Home shift code for the job. Used to identify a shift for use in pay calculations	String		85		
employee **> jobs** **> effectiveDatedJobInfo** **> shiftIndicator**	SHIFT_INDICATOR			Numeric identifier for the shift associated with this job	Integer				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee > jobs > effectiveDatedJobInfo > accrualStartDate	ASSIGNMENT_ACCRUAL_BEGIN_DATE			The date that is used to drive accrual calculations. Used for length of service in accrual bands.	Date			latest_hire_date	
employee > jobs > effectiveDatedJobInfo > latestStartDate	LATEST_ASSIGNMENT_BEGIN_DATE		StartDate	The date of last time this job started. This is used for rehires where the originalStartDate and latestStartDate may be different	Date				
employee > jobs > effectiveDatedJobInfo > isFullTime	FULLTIME_PARTTIME			An indicator for if this job is full-time. True if the job is full-time, false if the job is part-time	Boolean				
employee > jobs > effectiveDatedJobInfo > standardWeeklyHours	ASSIGNMENT_STD_WEEKLY_HOURS			Standard weekly hours for this job.	Number			5 * standardDailyHours	
employee > jobs >effectiveDatedJobInfo > standardDailyHours	ASSIGNMENT_STD_DAILY_HOURS			Standard daily hours for this job.	Number			8	
employee > jobs >effectiveDatedJobInfo > standardPeriodHours	ASSIGNMENT_STD_PERIOD_HOURS			The amount of hours per period that a person regularly works on this job	Number			8	
employee > jobs >effectiveDatedJobInfo > ftePercent	ASSIGNMENT_FTE_PCT			The employee's full-time equivalent (FTE) percentage in this job. This is a number between 0 and 100	Number	0 - 100			
employee > jobs > effectiveDatedJobInfo > payrollId	PAYROLL_EMP_ID		PayrollId	The job ID that matches the payroll system. This could be the same as the employee ID, depending on the payroll system	String		256	externalMatchId	

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee **> jobs** **>effectiveDatedJobInfo** **> PayrollSystemId**	PAYROLL_SYSTEM_ID			The system ID that corresponds to the payroll system. For example, there could be two payroll systems processing different types of time data.	String		2000		
employee **> jobs** **>effectiveDatedJobInfo** **> payType**	PAY_TYPE, PAY_RATE_TYPE			The customer-defined pay type for this specific job. Some examples would be salary or hourly	String	H | S	2000	H	
employee **> jobs** **> effectiveDatedJobInfo** **> employmentType**	EMPLOYMENT_TYPE			The customer-defined type of employment for this specific job. Some examples would be regular or temporary	String	R | T	2000		
employee **> jobs** **> effectiveDatedJobInfo** **> payGrade**	PAY_GRADE			Level on the pay scale for this job	String		2000		
employee **> jobs** **> effectiveDatedJobInfo** **> payCurrency**	PAY_CURRENCY			The ISO-4217 code identifying which currency the pay amounts for this job are represented in	Currency		3		
employee **> jobs** **>effectiveDatedJobInfo** **> policyProfile** (Note: Can this ever be supplied by a 3rd party HR system?)	POLICY_PROFILE			The profile that a job maps to. This is used to derive calculations in the system. It unlikely that this can be supplied by 3rd party HR but is provided as a convenience.	String		85		
employee **> jobs** **>effectiveDatedJobInfo** **> jobLevel**	JOB_LEVEL			Description of the level for the job	String		2000		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee > jobs >effectiveDatedJobInfo > flsaStatus	FLSA_STATUS			Status referring to the state of being exempt from the protection provided by the Fair Labor Standards Act	String		2000		
employee > jobs >effectiveDatedJobInfo > locationId	LOCATION_ID			Identifier for the location of this job	String		80		
employee > jobs >effectiveDatedJobInfo > company	COMPANY_ID			Identifier of the company for this job	String		2000		
employee > jobs >effectiveDatedJobInfo > department	DEPARTMENT_ID			Identifier of the department for this job	String		80		
employee > jobs >effectiveDatedJobInfo > district	DISTRICT_ID			Identifier of the district for this job	String		2000		
employee > jobs >effectiveDatedJobInfo > region	REGION_ID			Identifier of the region for this job	String		2000		
employee > jobs > effectiveDatedJobInfo > country	BUSINESS_ADDRESS_COUNTRY			The ISO 3 Character Country Code for this job, which is used to apply compliance templates	Country Code		60		
employee > jobs > effectiveDatedJobInfo > stateOrProvince	BUSINESS_ADDRESS_STATE_PROV			The work state/province for this job in ISO 3166-2 format, which is used to apply compliance templates.	State/Province		40		
employee > jobs	TIME_ZONE			IANA Time Zone Identifier for this job.	TZ		85	Based on US work state	*

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
>effectiveDatedJobInfo > timezone									
employee > jobs > effectiveDatedJobInfo > managersMatchValue	MANAGER_ID			Value indicating who this job reports to, which could be a manager's ID, a location, etc. This value is expected to match the value of the 'managedJobsMatchValue' field at the employee level, indicating that that employee is responsible for managing this job.	String		256		
employee > jobs > effectiveDatedJobInfo > timeSheetApproverMatchId	TIME_SHEET_APPROVER_MATCH_ID			Match ID of the approver of the job's timesheets	String		2000		
employee > jobs > effectiveDatedJobInfo > timeOffApproverMatchId	TIME_OFF_APPROVER_MATCH_ID			Match ID of the approver of the job's time off requests	String		2000		
employee > jobs > effectiveDatedJobInfo > importedBadgeId	IMPORTED_BADGE_ID			Imported badge ID used to generate a badge record when badge imports are run	String		2000		
employee > jobs > effectiveDatedJobInfo > unionCode	UNION_CODE			Identifier for the job's union	String		2000		
employee > jobs > effectiveDatedJobInfo > costCenter	COST_CENTER	{CERNER_SCHEDULE_MANAGEMENT}		The employee's cost center for this job.	String		80		
employee > jobs > effectiveDatedJobInfo	BUSINESS_ADDRESS_CITY	{BRA_EMP_IMPORT_MAPPINGS}{SCH_PREDICTIVE_SCHED}		Business Address city.	String		128		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
> businessAddress **> city**									
employee **> jobs** **> effectiveDatedJobInfo** **> businessAddress** **> lineOne**	BUSINESS_ADDRESS_LINE_1	{BRA_EMP_IMPORT_MAPPINGS}		Business Address Line 1.	String		60		
employee **> jobs** **> effectiveDatedJobInfo** **> businessAddress** **> lineTwo**	BUSINESS_ADDRESS_LINE_2	{BRA_EMP_IMPORT_MAPPINGS}		Business Address Line 2.	String		60		
employee **> jobs** **> effectiveDatedJobInfo** **> businessAddress** **> lineThree**	BUSINESS_ADDRESS_LINE_3	{BRA_EMP_IMPORT_MAPPINGS}		Business Address Line 3.	String		60		
employee **> jobs** **> effectiveDatedJobInfo** **> businessAddress** **> postalCode**	BUSINESS_ADDRESS_POSTAL_CODE	{BRA_EMP_IMPORT_MAPPINGS}		Business Address Line post code.	String		40		
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> applicableMinimumWage**	APPLICABLE_MINIMUM_WAGE	{US_WAGE_HOUR_AND_HOL}		The minimum wage which applies to this employee/job.	Number				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> assignmentMonthlyContractHours**	ASSIGNMENT_MTHLY_CONTRACT_HRS	{EXT_CONVERT_EL_DAYS_TO_EL_HOURS}		Employee's monthly contractual working hours used in the calculation of hours.	Number				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> branch**	BRANCH	{BRA_EMP_IMPORT_MAPPINGS}		Branch Company branch identifier.	String		2000		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> brazilEmployerId**	CEI	{BRA_EMP_IMPORT_MAPPINGS}		The Employer's Brazil employer identifier (CEI), if it exists.	String		2000		
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> daysPerMonth**	DAYS_PER_MONTH	{EXT_CONVERT_EL_DAYS_TO_EL_HOURS}		Employee's monthly contractual working days used in the calculation of hours.	Number				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> daysPerWeek**	DAYS_PER_WEEK	{DEU_STAT_VAC_BANK}{DEU_EMP_IMPORT_MAPPINGS}{EXT_CAN_VACATION_BANK_W_CARRY}{EXT_EU_ANNUAL_LEAVE}		The contracted number of days per week an employee works.	Integer				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> daysPerWeekFuture**	DAYS_PER_WEEK_FUTURE	{DEU_STAT_VAC_BANK}		The employee's number of work days per week as of a future effective date.	Integer				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> daysPerWeekFutureEffDt**	DAYS_PER_WEEK_FUTURE_EFF_DT	{DEU_STAT_VAC_BANK}		The ISO-8601-formatted date representing when the employee's future number of work days per week will become effective.	Date				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> employerName**	EMPLOYER_NAME_BRA	{BRA_EMP_IMPORT_MAPPINGS}		Name of the employer.	String		2000		
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> facilityIdentifier**	FACILITY_IDENTIFIER	{CERNER_SCHEDULE_MANAGEMENT}{CERNER_CLAIRVIA_SLICE_EXPORT_AND_PUNCH_EXPORT}		ID that determines the facility the home cost center belongs to.	String		2000		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> holidayCalendarMappingKey**	HOLIDAY_CALENDAR_MAPPING_KEY	{GLOBAL_BASE}		Optional field to use to map to the holiday calendar.	String		2000		
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> hourlyBaseRate** Mutually exclusive with salariedBaseRate	BASE_PAY_RATE	{EXT_CALLIN_MIN_USER_IND}		Explicit hourly rate for hourly employees.	Number				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isAlternativeSchedule**	ALTERNATIVE_SCHEDULE_INDICATOR	{CGT_CAN_AB_FLEX_AVG }		Flag to indicate that this employee is using an alternative schedule.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isCasual** Works in conjunction with the 'isFullTime' flag to set the EMPLOYEE_TYPE field to one of the following values: "Casual", "Full-Time", "Part-Time"	EMPLOYEE_TYPE	{AUS_RETAIL_AWARD_OT_PREMIUM} {AUS_RETAIL_AWARD_HOL_PREM} {AUS_RETAIL_AWARD_PENALTY} {AUS_RETAIL_AWARD_SHIFT_PREM}		Flag to indicate whether this employee has the 'Casual' status.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isExcludedFromOtc**	EXCLUDE_FROM_OTC	(none)		Flag to indicate whether this employee should be excluded from One Touch Callouts.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isExemptFromOTCap**	HIGHLY_PAID_WORKER	{JPN_CAP_ON_OT}		Flag to identify workers that are exempt from the cap on overtime hours.	Boolean				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isPunchingForBreaks**	PUNCH_FOR_BREAKS_FLAG	{BRA_EMP_IMPORT_MAPPINGS}		Flag to indicate whether an employee punches for breaks or not.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isQualifiedForSickLeaveAccrual**	SICK_LEAVE_ACCRUAL_QUAL	{STANDARD_PRODUCT}		Flag to indicate eligibility for sick bank accruals.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isQualifiedForVacationAccrual**	VACATION_ACCRUAL_QUALIFICATION	{STANDARD_PRODUCT}		Flag to indicate eligibility for vacation bank accruals.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isRehire**	REHIRE_INDICATOR	{CGT_US_SICK_LEAVE} {CGT_US_SICK_LEAVE_CA} {CGT_US_SICK_LEAVE_DC} {CGT_US_SICK_LEAVE_JRSY_CTY} {CGT_US_SICK_LEAVE_LA} {CGT_US_SICK_LEAVE_MA} {CGT_US_SICK_LEAVE_MG} {CGT_US_SICK_LEAVE_NWRK} {CGT_US_SICK_LEAVE_NYC} {CGT_US_SICK_LEAVE_OR} {CGT_US_SICK_LEAVE_PHIL} {CGT_US_SICK_LEAVE_SAN_DG} {CGT_US_SICK_LEAVE_SAN_FRAN}		Flag to identify the employee as a rehire to support validation and messaging when rehire conditions may apply.	Boolean				

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
		{CGT_US_SICK_LEAVE_SEA} {AUS_LSL}							
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isShiftWorker**	SHIFT_WORKER	{AUS_RETAIL_AWARD_HOL_PREM} {AUS_RETAIL_AWARD_PENALTY} {AUS_RETAIL_AWARD_SHIFT_PREM}		Shift Workers receive the same options for the shift that start on the holiday, they do not receive shift premium for holidays.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isTaking30MinuteBreaks**	BREAK_30_MINUTE_FLAG	{UK_WTD_DAILY_REST} {UK_WTD_DATA_ELEMENTS} {CGT_PR_LAWS_MEAL}		Flag that indicates whether the employee agrees to reduce his/her meal periods to 30 minute.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isUsingFlexiTime**	FLEXI_TIME_INDICATOR	{DEU_EMP_IMPORT_MAPPINGS} {EXT_FLEXI_TIME}		Flag that indicates whether an employee is working under the flexi time regulations.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isWeekendsOnly**	WEEKEND_ONLY	{WEEKEND_DAYS_OFF}		Flag that designates that the employee only works weekends.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> isWorkingIrregularSchedule**	IRREGULAR_SCHEDULE_FLAG	{EXT_HOL_PAY_ELIGIBILITY}		Flag to identify employees who work on irregular schedules.	Boolean				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> miniJobberMidiJobber**	MINI_JOBBER_MIDI_JOBBER	{DEU_EMP_IMPORT_MAPPINGS}		A code that indicates whether an employee is a Mini Jobber (MN) or a Midi Jobber (MD).	String		2000		
employee **> jobs** **> effectiveDatedJobInfo**	CNPJ	{BRA_EMP_IMPORT_MAPPINGS}		Brazil employer's National Registry of Legal Entities (CNPJ) identifier.	String		2000		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
> wtaTemplateJobFields **> nationalRegistryOfLegalEntitiesId**									
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> programOfSocialIntegrationId**	PIS	{BRA_EMP_IMPORT_MAPPINGS}		The employee's Program of Social Integration (PIS) identifier.	String		2000		
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> policyProfileMappingKey**	POLICY_PROFILE_MAPPING_KEY	{GLOBAL_BASE}		Optional field to use to map to policy profile.	String		2000		
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> salariedBaseRate** Mutually exclusive with hourlyBaseRate	BASE_PAY_RATE	{EXT_CALLIN_MIN_USER_IND}		Explicit annual salary for salaried employees.	Number				
employee **> jobs** **> effectiveDatedJobInfo** **> wtaTemplateJobFields** **> skillCode**	SKILL_CODE	{CERNER_SCHEDULE_MANAGEMENT}{CERNER_CLAIRVIA_SLICE_EXPORT_AND_PUNCH_EXPORT}		Identifies the employee's skill code.	String		2000		
employee **> jobs** **> effectiveDatedJobInfo** **> wfandsJobFields** **> contractType**			ContractType	The contract type of this employee. Also known as Position in Scheduling and mapped to work profiles.	String	Usually 'CASUAL', 'PARTTIME' etc. but list is configurable	16		
employee **> jobs** **> effectiveDatedJobInfo** **> competencies** **> competencyExternalId**			Most common usage is: Task	ID that uniquely identifies a competency within the customer's system. The most common usage for Competencies are to assign tasks to employees in the Scheduling application. In that scenario, the competencyExternalId	String		128		*

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
				must exactly match the Task Code configured in the WorkForce Scheduling application.					
employee **> jobs** **> effectiveDatedJobInfo** **> competencies** **> competencyType**				A categorisation for the competency.	Enum	TASK | SKILL | LICENSE | QUALIFICATION			*
employee **> jobs** **> effectiveDatedJobInfo** **> competencies** **> schedulingUnitRestrictions** **> schedulingUnitExternalId**				The external identifier for the scheduling unit. Note: must match the scheduling unit code exactly in the Scheduling application. For tasks, there must also be a corresponding schedulingUnitExternalId in the subsequent schedulingUnits section.	String		64		*
employee **> jobs** **> effectiveDatedJobInfo** **> competencies** **> schedulingUnitRestrictions** **>isDefault**				Used to indicate if the task (competency) being assigned to an employee in a specific scheduling unit should be the primary task in that specified scheduling unit.	Boolean				*
employee **> jobs** **> effectiveDatedJobInfo** **> schedulingUnits** **> schedulingUnitExternalId**				The external identifier for the scheduling unit. Note: must match the scheduling unit code exactly in the Scheduling application.					*
employee **> jobs**				Flag to indicate whether this is default	Boolean				*

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
> effectiveDatedJobInfo > schedulingUnits > isPrimarySchedulingUnit				scheduling unit for the employee.					
employee > jobs > effectiveDatedJobInfo > payRates > rate			PayRate	This field is specific to the Scheduling application and used to pass the pay rate into that system for schedule costing calculation purposes.	String				
employee > jobs > effectiveDatedJobInfo > payRates > frequency			PayRateType	This field is specific to the Scheduling application and used to pass the pay rate type/frequency into that system for schedule costing calculation purposes.	String	HOURLY, DAILY, WEEKLY, FORTNIGHTLY, MONTHLY, YEARLY			
employee > jobs > effectiveDatedJobInfo > payRates > payGroup			PayGroup	This field is specific to the Scheduling application and used to pass the pay group into that system for schedule costing calculation purposes.	String		32		
employee > jobs >effectiveDatedJobInfo > customFields > key => value				Map of key/value pairings defining additional attributes that apply to this job.	String => String		Varies		
user	The object representing the entire set of user data for the person. **Required if employee is not supplied**								
user > loginId	EXTERNAL_HR_ID / AUTHENTICATION_MATCH_ID		UserName	Login id which is unique within the customer's account	String		80		*
user > ssoMatchId	AUTHENTICATION_MATCH_ID			ID that is used for user matching specifically within the context of Single Sign On (SSO) authentication	String		256		

Schema / API field name	T&A data element	T&A template	F&S (staff) import	Description	Format	Valid values	Length	Default Value	Required
user **> timezone**	TIME_ZONE			IANA Time Zone Identifier for this user	TZ		85		
user **> enabled** Negates the value sent in (if they aren't enabled, they are locked out)	LOCKED_OUT			If an enabled this user can authenticate	Boolean				*
user **> localePolicy**	LOCALE_POLICY			Locale that maps to the time and attendance locale policy	String		85		
user **> allowUnlimitedNonBiometricLogins**	UNLIMITED_NON_BIO_LOGINS			Indicates if this user is allowed unlimited non-biometric logins. Defaults to false	Boolean				
user **> managedSchedulingUnits** **> msu**			JobRole	A list of "divisions" that can be managed (equivalent to managed divisions in F&S) by a manager who may be a user and may or may not be scheduled or have timesheets	String		64		
user **> wfandsUserFields** **> role**			JobRole	The persons role	Enum	STAFF | MANAGER | MANAGER_STAFF			
user **> wfandsUserFields** **> accessLevel**			AccessLevel	The access level to be given to the employee's user.	String		32		
user **> customFields** **> key => value**				Map of key/value pairings defining additional attributes that apply to this user.	String => String		varies		

Appendix B: Frequently Asked Questions (FAQs)

Q1: F how long is the Authorization Token good, and what happens when it expires?

A1: As mentioned, you will need to get an Authorization Token through the Authorization Service and use that token when making calls to the APIs. The token has a finite life and will expire, after which you will need to obtain a new token. You can see the expiration time in “expires_in” value in the response of the service. If your token expires while your API is executing, the API will finish processing before it times out. However, the next request to the API will need to have a new token. The application that is making the API calls should trap the expiration and renewal of the tokens as part of the logic that is using the REST APIs.

Q2: How much data is returned in the Data Feed APIs?

A2: The default number of records to return is 2,000 and the maximum that can be returned in one call to the Data Feed APIs is 10,000 records. The application making the calls to the API should use the count in the parameters of their request and loop through the cursors and make subsequent calls to retrieve the required data.

Q3: When the last set of data is retrieved by the Data Feed will the cursor be blank?

A3: When the next pull of the data happens, the cursor will always come back. When you are at the end, the next cursor is what you would use to get the next set of data. If you try to get the next cursor, it could come back with zero rows until you call it again and there is resulting data to be retrieved. You would check to see if there is an empty array and check to see if the cursor had changed.

Q4. Is there a way to get the total record count (i.e., select count(*) from employee)?

A4: No, because the API isn't like calling a SQL statement, it is a feed of changes that have been captured (i.e., calculated time).

Q5. Is there a way to search for data based on some criteria (i.e., employee id)?

A5. The majority of Data Feed APIs are bulk extracts of data that will provide downloads of data sets. There isn't any "where clause". However, we do have a few APIs which will allow you to pass some parameters to target the data you are trying to retrieve. We have versions of the GET "Calculated Time Employee", “Timesheet Summary” and “Person” APIs that have some criteria you could filter on (i.e., externalMatchID, toDate, fromDate, etc) to get data just for one employee.

Q6. Is the data from the Data Feed stored in the source database or in the data bus?

A6. The data is accessed via the database that holds the data in the data bus. They do not reach into the source of the data (i.e., WTA) when the API is called.

Q7. How do we reset the data or start over?

A7. You could call the API with no cursor and it will start to pull the data down again from the beginning. You would then follow the same process of looping through the cursors to download all of the data.

Q8. What is the recordID of the Data Feed API response?

A8. It is unique id for the record that was retrieved. If there was data before for an ID and later the data was removed from the timesheet, the next pull for that recordID would have blank data indicating that the data is deleted (with the delete flag). The ID is a hashed value from a concatenation of different fields.